MA124 Maths by Computer

Project: Constructing and applying Machine Learning models

Background

Machine Learning is the use and development of computer systems that are able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyse and draw inferences from patterns in data. In machine learning we are interested in creating functions or models which fit real data. Such functions can be found by making them dependent on a number of parameters which are updated over and over again to improve the fit to the data. You will explore in this project.

Suggested reading

There are many great articles, books, videos about machine learning. This list just picks out a few:

Linear Regression Model. Short, simple article covering a task similar to A2 below.

Logistic Regression Model An article which covers models similar to those in A3 and A4 below.

What is a neural network. Great YouTube series of introductory videos about training neural networks by the excellent 3Blue1Brown, well worth watching.

Blog: An Introduction to Neural Networks. Good discussion of the basic ideas used in this project.

Hands on Machine Learning with Python This might accompany a full course on machine learning. It has much more detail that you will need. As the title says, it's all done using Python. It's well-written and popular.

Struture of project

There are 6 tasks in the document below, tasks A1-A4 in Section A and then tasks B1 and B2 in section B. Your group should do all the tasks in section A and at least one of the two tasks in section B. A1-A4 are worth approximately 60% of the credit for this submission and section B is worth approximately the other 40%.

Notes on submission

Read through the document **MA124 Maths by Computer Tutor Group Projects Information for Students.pdf** on the MA124 Moodle page.

Before submitting see the notes at the end of this document.

Allowed libraries for this project:

numpy matplotlib copy pandas scikitlearn. **Note you may not use scikitlearn in any of tasks A2, A3 or A4.**

This means that you may include these lines of code at the start of of your code cells (and please use the aliases given below).

In [134…

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import copy
import sklearn # Not to be used in any of tasks A2, A3 or A4.
```

Section A - (worth approximately 60% of the marks)

Task A1 - Linear Regression machine learning task using SciKitLearn (worth approximately 20% of the marks)

In a recent research article published in the journal Computer Communications, authors Sathishkumar V E, Jangwoo Park, and Yongyun Cho sought to predict the "bike count required at each hour for the stable supply of rental bikes"[1]. They employed several regression models, including linear regression. The dataset used in the original study is available here.

Task: Apply machine learning to a modified version of the original dataset and report the results.

[1] Sathishkumar V E, Jangwoo Park, and Yongyun Cho. 'Using data mining techniques for bike sharing demand prediction in metropolitan city.' Computer Communications, Vol.153, pp.353-366, March, 2020 web link.

The original research article and a modified dataset are posted on the MA124 Moodle page in the resources for this project. You will need to refer to the article for some of the tasks below. You will need to download SeoulBikeData_mod.csv and put it into the folder with this file.

SeoulBikeData_mod.csv has been modified from the original dataset to remove the categorical variables, and to convert dates to months. Months have been coded by number, e.g. 1 = January, etc. Only half the months are included in the modified dataset.

While the number of tasks is large, this is in part because the instructions are rather specific. Many of them are very similar to things you saw in term 1, in the week 8 and week 10 notebooks and lectures.

Specifically, your code should

1. Import needed libraries. (You will need pandas, as well as things from sklearn, and of course numpy and matplotlib.)

2. Using pandas, read SeoulBikeData_mod.csv into a Dataframe.

3. `describe` the Dataframe.

4. Plot a histogram of `Rented Bike Count`. Do not plot this as a density, but as a count. See Fig. 3 of the article. The vertical axis in the article is labelled "frequency", but is the same as the count.

   Produce a box plot similar to that in Fig. 3 of the article.

   Try to generate both the histogram and box plot to look approximately as they do in the article.

5. From the full Dataframe, create a new Dataframe `X` containing all the columns except `Rented Bike Count` and a Series `y` containing only the `Rented Bike Count` column. These are your design matrix and target respectively.

6. Perform a test-train split to create `X_train`, `X_test`, `y_train` and `y_test`. You **must** use the same percentage of data for testing and training as was used in the article and you **must** state what they are. You can find these in the article.

7. Create and train a linear regression model.

8. Use the trained model to obtain `y_pred`, the prediction on the test data `X_test`. Form the residual `resid = y_test - y_pred`.

9. Compute and report: Rsquared (R2), the Root Mean Squared Error (RMSE), the Mean Absolute Error (MAE), and the Coefficient of Variation (CV). Compare these results to those on the top, right of Table 4 of the article. (Note, the modified dataset we are studying is different from that used in the article. Hence the results will not be identical. However, the procedure is very close to that used in the article.)

10. Produce and comment on the following plots.

- Histograms of `y_test` and of `y_pred` (on the same plot). These should be reported as counts rather than densities.

- A scatter plot of `resid` as a function of `y_test` corresponding to Fig. 9 of the article. (Recall what `y_test` represents and label the plot appropriately.) Unlike Fig. 9 of the paper, you should use a colormap to plot the different `Hours` in different colours.

- A scatter plot of `resid` as a function of `X_test['Month']`. Use a colormap to indicate the absolute value of `resid`.

- A scatter plot of `resid` as a function of `X_test['Rainfall(mm)']`. Use a colormap to indicate the absolute value of `resid`.

Insert code and markdown cells below in which to answer this.

1. Importing libraries

In [135…
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_e
```

2. Uploading the dataset

In [136…
```
#2

# bike_data = pd.read_csv("/content/SeoulBikeData_mod.csv")
bike_data = pd.read_csv("Seoul Bike Data.csv")
```

3. Print the statistical data of all columns in the dataset.

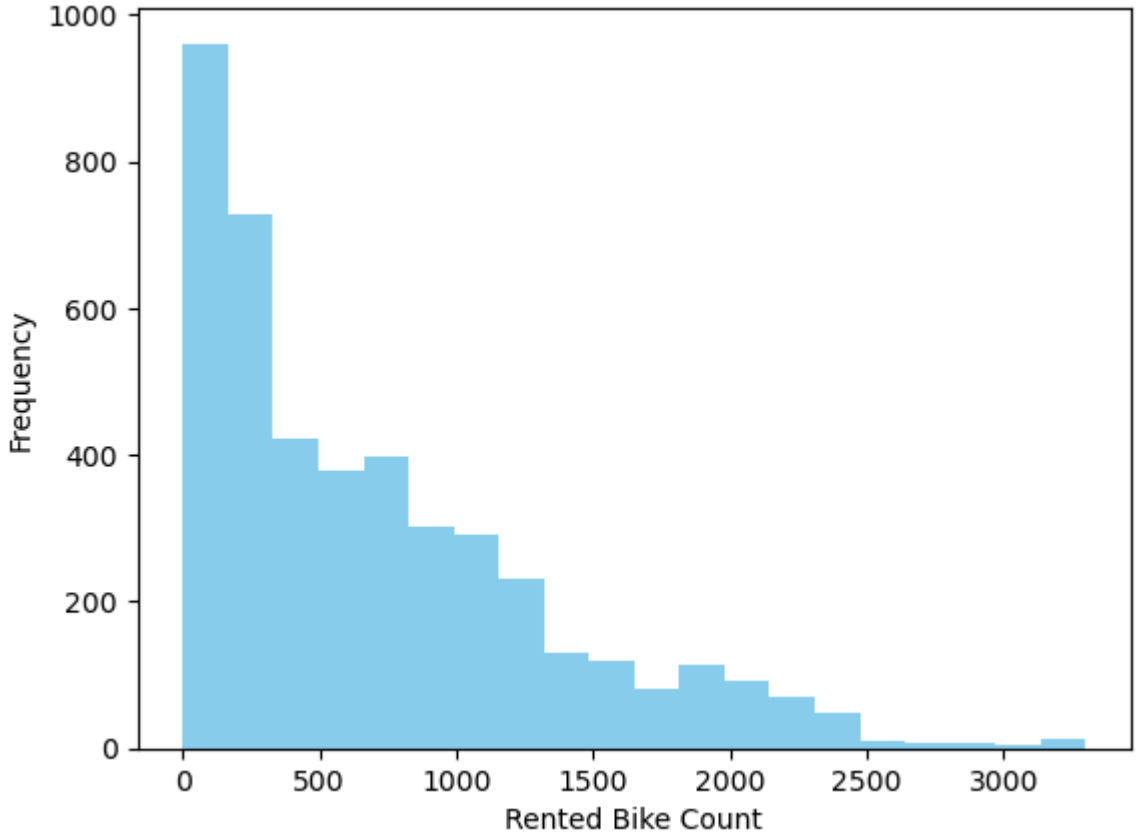
In [137…
```
#3
print(bike_data.describe())
```

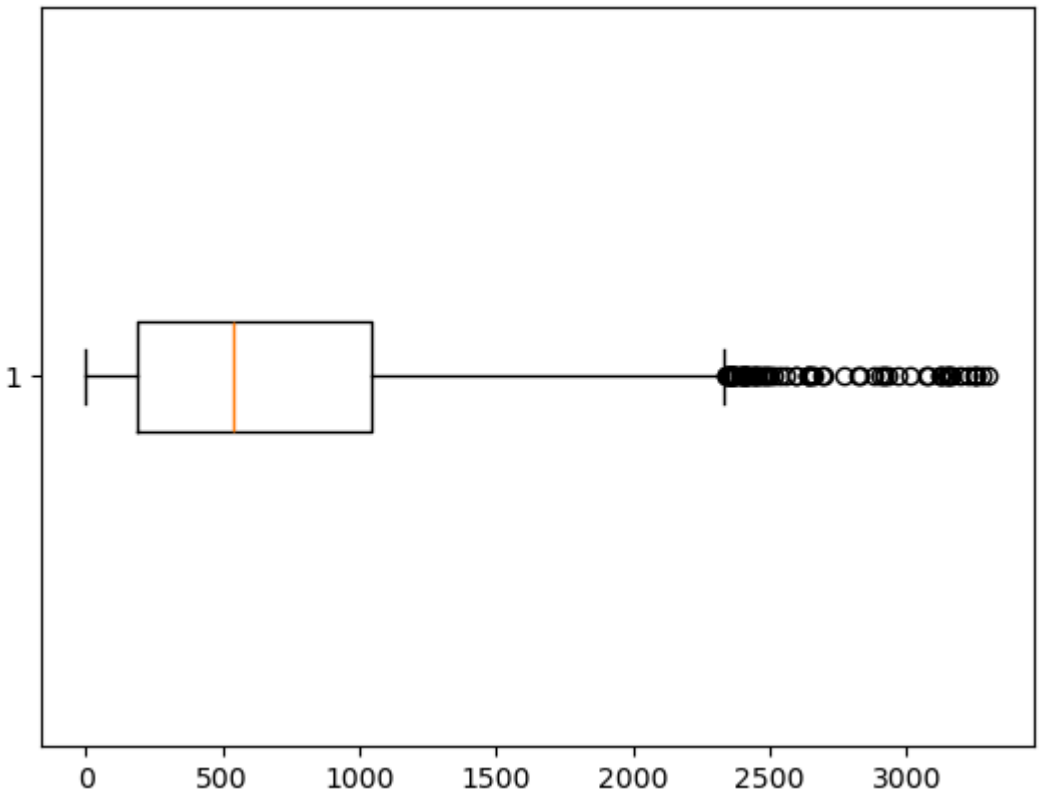
```
       Rented Bike Count        Hour  Temperature(°C)  Humidity(%)  \
count        4416.000000  4416.00000      4416.000000  4416.000000
mean          704.591259    11.50000        13.203986    59.257246
std           637.407244     6.92297        11.529724    20.162981
min             0.000000     0.00000       -17.800000     0.000000
25%           189.000000     5.75000         4.500000    44.000000
50%           542.000000    11.50000        14.300000    58.000000
75%          1047.000000    17.25000        22.200000    75.000000
max          3298.000000    23.00000        38.000000    98.000000

       Wind speed (m/s)  Visibility (10m)  Dew point temperature(°C)  \
count       4416.000000       4416.000000                4416.000000
mean           1.653193       1398.641304                   4.723958
std            0.976361        621.560536                  12.671445
min            0.000000         27.000000                 -30.600000
25%            0.900000        853.000000                  -3.500000
50%            1.500000       1621.000000                   6.400000
75%            2.200000       1999.000000                  14.900000
max            6.700000       2000.000000                  26.800000

       Solar Radiation (MJ/m2)  Rainfall(mm)  Snowfall (cm)        Month
count              4416.000000   4416.000000    4416.000000  4416.000000
mean                  0.560181      0.139312       0.070879     5.956522
std                   0.860496      1.145271       0.437416     3.407262
min                   0.000000      0.000000       0.000000     1.000000
25%                   0.000000      0.000000       0.000000     3.000000
50%                   0.010000      0.000000       0.000000     5.000000
75%                   0.910000      0.000000       0.000000     9.000000
max                   3.520000     35.000000       8.800000    11.000000
```

Plotting a histogram of

In [138…

```
#4
plt.hist(bike_data["Rented Bike Count"], color = "skyblue", bins = 20)
plt.xlabel('Rented Bike Count')
plt.ylabel('Frequency')
plt.show()
plt.boxplot(bike_data["Rented Bike Count"], vert = False)
plt.show()
```





In [139…

```
#5
X = bike_data.drop("Rented Bike Count", axis = 1)
Y = bike_data["Rented Bike Count"]
```

In [140…

```
#6
#75% - training, 25% - testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    Y,
    test_size=0.25,
    random_state=42
)
```

In [141…

```
#7
model = LinearRegression()
model.fit(X_train, y_train)
```

Out[141…

```
▾ LinearRegression
LinearRegression()
```

In [142…

```
#8
y_pred = model.predict(X_test)
resid = y_pred - y_test
```

In [143…

```
#9
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
cv = np.std(resid)/np.mean(resid)

print(f"R-squared: {r2}")
print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
print(f"CV: {cv}")
```

```
R-squared: 0.41463066046384467
RMSE: 476.04344275468833
MAE: 343.86701600340734
CV: 87.51078074972355
```

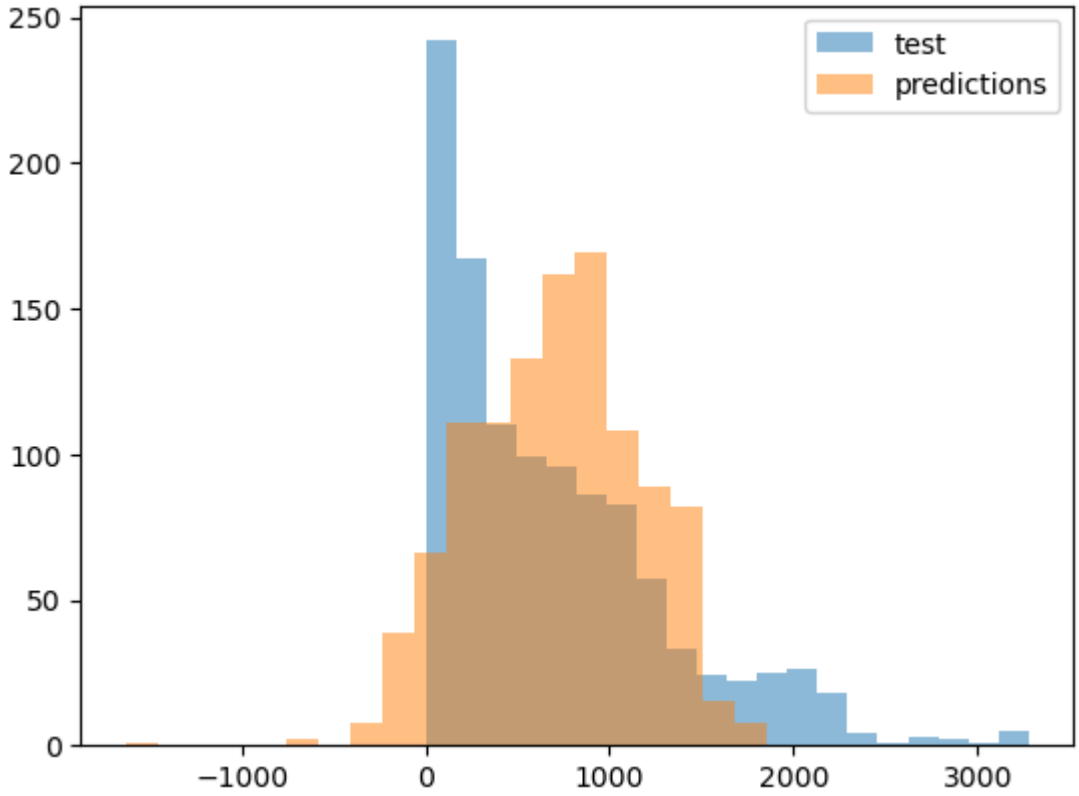
10. Produce and comment on the following plots.

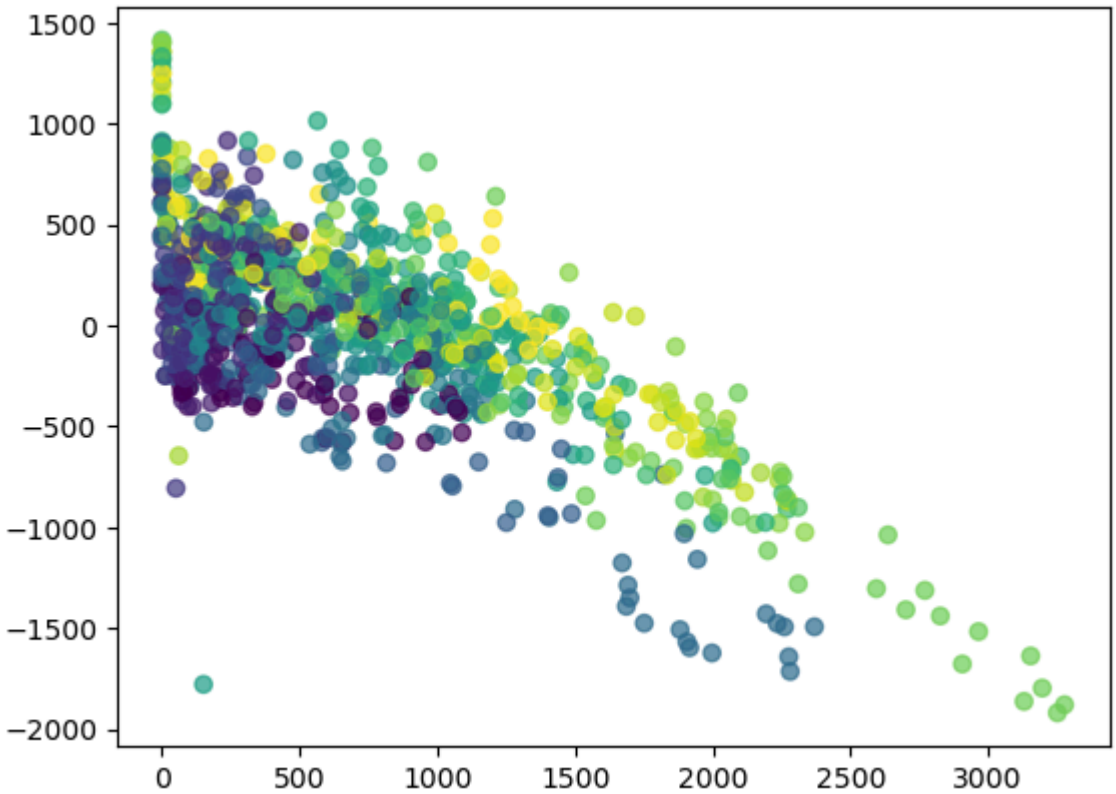
- Histograms of `y_test` and of `y_pred` (on the same plot). These should be reported as counts rather than densities.
- A scatter plot of `resid` as a function of `y_test` corresponding to Fig. 9 of the article. (Recall what `y_test` represents and label the plot appropriately.) Unlike Fig. 9 of the paper, you should use a colormap to plot the different `Hours` in different colours.
- A scatter plot of `resid` as a function of `X_test['Month']`. Use a colormap to indicate the absolute value of `resid`.
- A scatter plot of `resid` as a function of `X_test['Rainfall(mm)']`. Use a colormap to indicate the absolute value of `resid`.

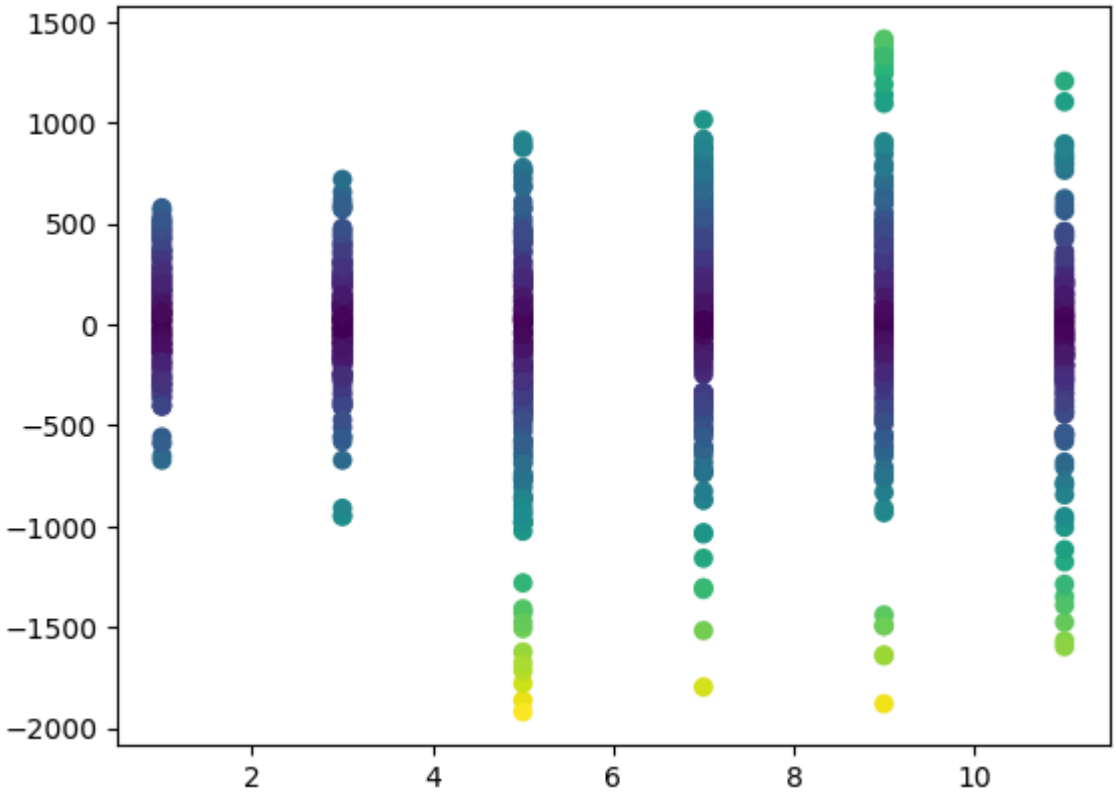
In [144…

```
#10
plt.hist(y_test, alpha = 0.5, bins = 20, label = "test")
plt.hist(y_pred, alpha = 0.5, bins = 20, label = "predictions")
plt.legend()
plt.show()
```

```
plt.scatter(y_test, resid, c = X_test["Hour"], alpha = 0.7)
plt.show()
plt.scatter(X_test["Month"], resid, c = abs(resid))
plt.show()
plt.scatter(X_test['Rainfall(mm)'], resid, c = abs(resid))
```

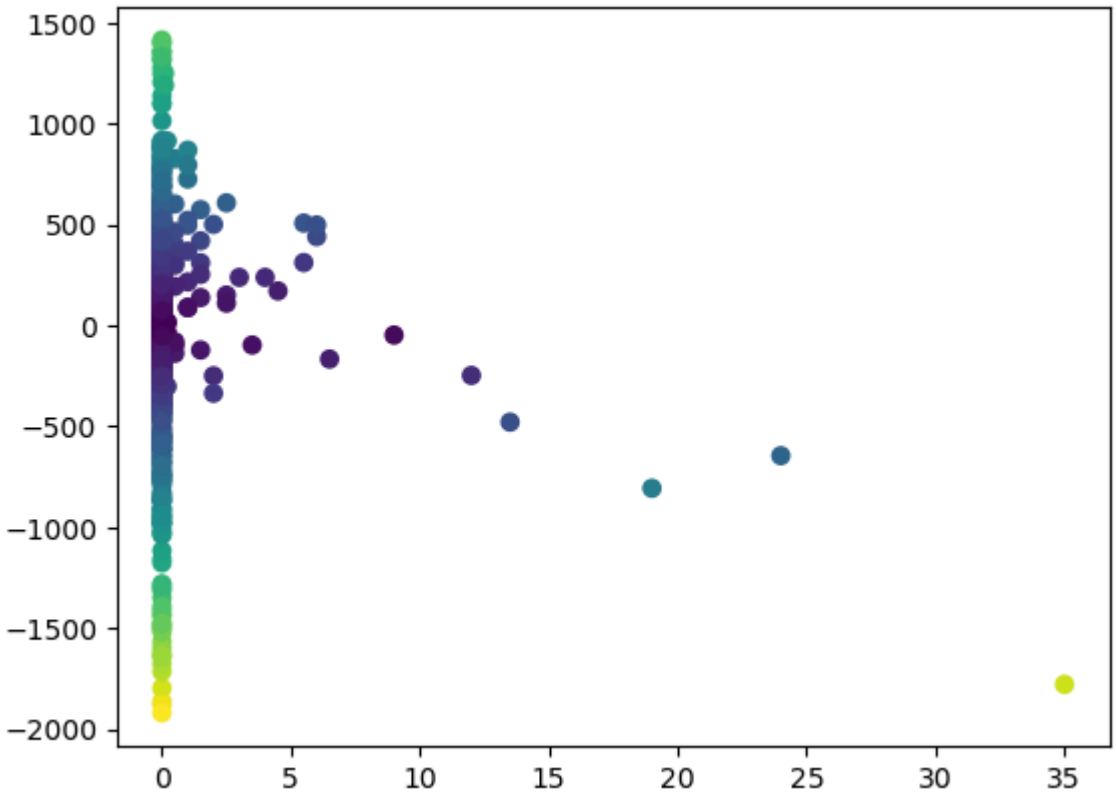






Out[144…

```
<matplotlib.collections.PathCollection at 0x16c4c8550>
```



In [ ]:

B2

Decision Trees

Rather than looking for linear dependencies (like Linear Regression) a Decision Tree splits the data into subsets based on the "if-then" principle, for instance "if 12 < Hour < 17 and Temperature > 15, then predict 300 bikes".

The code for the implementation:

In [145…

```
tree_model = DecisionTreeRegressor(max_depth=3, random_state=42)
tree_model.fit(X_train, y_train)
tree_pred = tree_model.predict(X_test)
tree_resid = tree_pred - y_test


tree_r2 = r2_score(y_test, tree_pred)
tree_rmse = np.sqrt(mean_squared_error(y_test, tree_pred))
tree_mae = mean_absolute_error(y_test, tree_pred)
tree_cv = np.std(tree_resid)/np.mean(tree_resid)

print(f"R-squared: tree: {tree_r2}, lin. regression: {r2}")
print(f"RMSE: tree: {tree_rmse}, lin. regression: {rmse}")
print(f"MAE: tree: {tree_mae}, lin. regression: {mae}")
print(f"CV: tree: {tree_cv}, lin. regression: {cv}")
```

```
R-squared: tree: 0.5453713663774171, lin. regression: 0.41463066046384467
RMSE: tree: 419.5270752147657, lin. regression: 476.04344275468833
MAE: tree: 288.27962653232936, lin. regression: 343.86701600340734
CV: tree: 100.63305032056786, lin. regression: 87.51078074972355
```

Decision trees can be considered a better model in this scenario because they capture non-linear patterns much better than linear regrression, which is very useful in this model because whilst the bike count is less when it is cold, it also decreses when it gets too hot to ride.

However, decision trees themselves tend to overfit, producing less acurate predictions, and also exptrapolate very poorly as they only set the conditions for the data in the given range. As the result, the above model performs worse in the root square test.

Whilst there is a sufficiently big dataset covering a large range of values, implying that there are very few anomalies for which the model actually needs to extrapolate, overfitting still remains a problem.

Random Forest

In order to minimise the variance caused by the overfitted model, a random forest model can be used, which takes many decision trees and finds the average.

In [146…

```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(n_estimators=100, max_depth=3, random_st
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_resid = rf_pred - y_test
```

```
rf_r2 = r2_score(y_test, rf_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_cv = np.std(rf_resid)/np.mean(rf_resid)

print("Random Forset Results")
print(f"R-squared: {rf_r2}")
print(f"RMSE: {rf_rmse}")
print(f"MAE: {rf_mae}")
print(f"CV: {rf_cv}")
```

```
Random Forset Results
R-squared: 0.5694882752108892
RMSE: 408.24802614578505
MAE: 279.61990437603515
CV: 232.34972566090815
```

XGBoost

The model can be further improved by using XGBoost algorithm. It also uses many decision trees like Random Forest, but whilst Random Forest trains on many random decision trees, xgboost targets the errors in every next decision tree focusing on minimising the errors.

Note that for the below code to run, xgboost must be installed on your system. This can be done with:

- pip install xgboost

In [147…

```
from xgboost import XGBRegressor

xgb_model = XGBRegressor(n_estimators=100,learning_rate=0.1,max_depth=3,r

xgb_model.fit(X_train, y_train)

xgb_pred = xgb_model.predict(X_test)

xgb_r2 = r2_score(y_test, xgb_pred)
xgb_rmse = np.sqrt(mean_squared_error(y_test, xgb_pred))
xgb_mae = mean_absolute_error(y_test, xgb_pred)

print(f"XGBoost R-squared: {xgb_r2:.2f}")
print(f"XGBoost RMSE: {xgb_rmse:.2f}")
print(f"XGBoost MAE: {xgb_mae:.2f}")
```

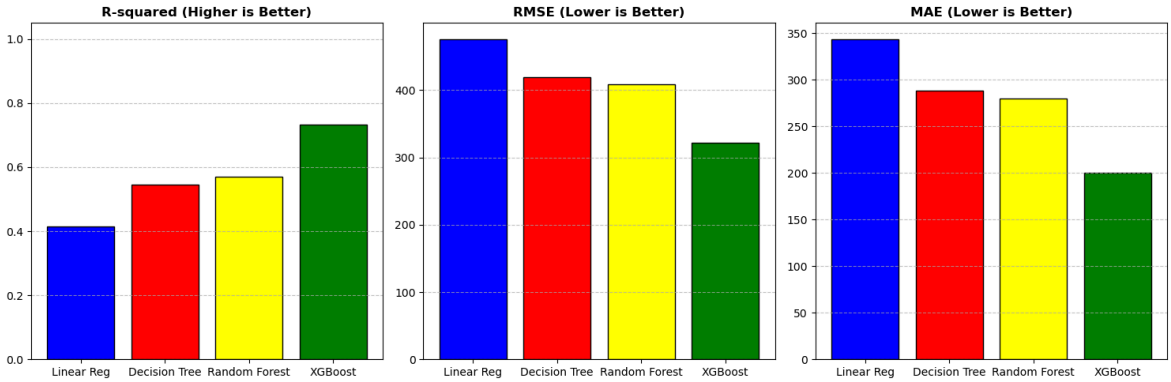
```
XGBoost R-squared: 0.73
XGBoost RMSE: 321.58
XGBoost MAE: 199.89
```

Test comparisons

In [148…

```
import matplotlib.pyplot as plt

models = ['Linear Reg', 'Decision Tree', 'Random Forest', 'XGBoost']

r2_scores = [r2, tree_r2, rf_r2, xgb_r2]
rmse_scores = [rmse, tree_rmse, rf_rmse, xgb_rmse]
```

```
mae_scores = [mae, tree_mae, rf_mae, xgb_mae]

colors = ['blue', 'red', 'yellow', 'green']

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].bar(models, r2_scores, color=colors, edgecolor='black')
axes[0].set_title('R-squared (Higher is Better)', fontsize=12, fontweight
axes[0].set_ylim(0, 1.05) # Cap R2 at slightly above 1.0
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

axes[1].bar(models, rmse_scores, color=colors, edgecolor='black')
axes[1].set_title('RMSE (Lower is Better)', fontsize=12, fontweight='bold
axes[1].grid(axis='y', linestyle='--', alpha=0.7)

axes[2].bar(models, mae_scores, color=colors, edgecolor='black')
axes[2].set_title('MAE (Lower is Better)', fontsize=12, fontweight='bold'
axes[2].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```



Task A2 Linear Regression, on a very small dataset, from scratch with only numpy (worth approximately 10% of the marks)

(illustrating idea of rates of change with respect to the model parameters and gradient descent)

Consider a really small dataset consisting of only three points in the plane: $(1,1),(2,4),(3,5)$.

For each point (training example) consider the x-coordinate as a single input feature, and the y-coordinate as the output or label of the example. The objective is to find the best line to fit these data. Here 'best' will mean the line which minimises the mean of the sum of the squares of the residuals with respect to the three points.

In this task and in the following tasks A3 and A4 we will use a slightly different convention to store our data with respect to the one used in A1 or the lectures. X_{train} and y_{train} will now be the transpose of the corresponding matrices in A1. In A3 and A4 this will be a more convenient form for applying functions given by matrices to the data with the matrix on the left of the data matrix.

The feature matrix X_{train} contains the feature vectors for all the training examples. More precisely $X_{\text{train}}= \begin{pmatrix} 1 & 2 & 3 \end{pmatrix} $ has one column for each training example and each column is the feature vector for that training example (in this case we have only one feature which is the x-coordinate).

The labels matrix y_{train} contains the labels for all the training examples. More precisely $y_{\text{train}} = \begin{pmatrix} 1 & 4 & 5\end{pmatrix}$ has one column for each training example and each column contains the label for that training example (the y-coordinate).

The model here is a simple linear model

$$ \hat y = mx +c $$

with trainable parameters m and c. The goal is to find the optimal values of m and c to fit these training data. In the Machine Learning context this means defining a loss function between prediction and true label:

$$ L(\hat y, y)=(\hat y - y)^2 $$

and a cost function which is simply the average on all training examples of these losses.

Specifically, you should

1. Show, in a markdown cell, how the cost J depends only on the trainable parameters m and c, and can be computed to be: $$ J=\displaystyle\frac{14m^2+12cm-48m+3c^2-20c+42}{3}$$

2. Find, in a markdown cell, expressions for the partial derivatives $\displaystyle\frac{\partial J}{\partial m}$ and $\displaystyle\frac{\partial J}{\partial c}$.

3. Write a function `model1(alpha, num_iterations)` which takes as inputs a learning rate `alpha` and a number of iterations `num_iterations`, and returns optimized values of m and c through Gradient Descent. More precisely the function should initialize m and c randomly between -2 and 2 and then perform `num_iterations` steps of gradient descent with learning rate `alpha`. This means that the value of m is updated to $m - \displaystyle\frac{\partial J}{\partial m}\alpha$ and the value of c is updated to $c - \displaystyle\frac{\partial J}{\partial c}\alpha$ in each iteration. Your function should print the cost J periodically throughout the iteration process. You may wish to refer to the Machine learning project lecture for help with this. You may wish to build helper functions for the various tasks you need this function to do and refer to them.

4. Plot the 3 datapoints along with the optimal line $mx+c$ your model has found.

5. Explore the effect of changing the learning rate and the number of iterations.

Insert code and markdown cells here in which to answer this task

1. $$J=\displaystyle\frac{L(\hat{y}_{1},{y}_{1})+L(\hat{y}_{2},{y}_{2})+L(\hat{y}_{3},{y}_{3})}{3}$$ $$J=\displaystyle\frac{L(m+c,1)+L(2m+c,4)+L(3m+c,5)}{3}$$ $$J=\displaystyle\frac{(m+c-1)^2+(2m+c-4)^2+(3m+c-5)^2}{3}$$ $$J=\displaystyle\frac{14m^2+12cm-48m+3c^2-20c+42}{3}$$
2. $$\displaystyle\frac{\partial J}{\partial m}=\displaystyle\frac{28m+12c-48}{3}$$ $$\displaystyle\frac{\partial J}{\partial c}=\displaystyle\frac{12m+6c-20}{3}$$

In [149…

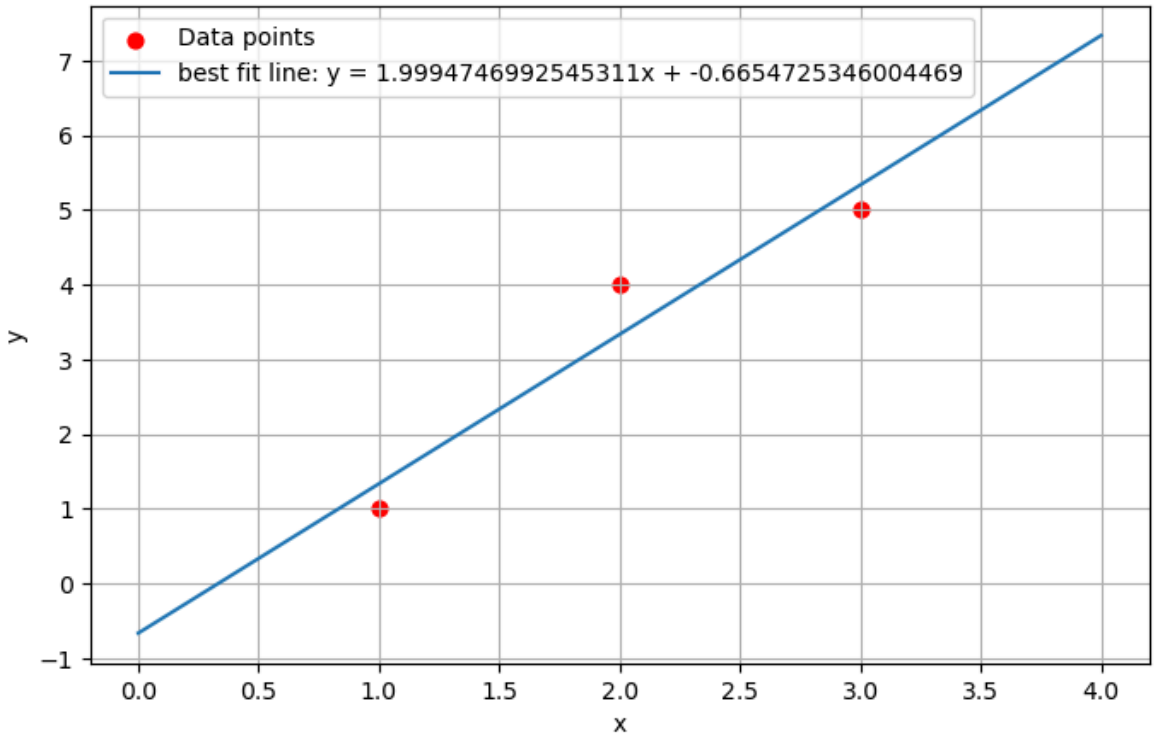
```
import numpy as np
import matplotlib.pyplot as plt

def model1(alpha, num_iterations):
    #random m and c in [-2,2]
    m = np.random.uniform(-2, 2)
    c = np.random.uniform(-2, 2)
    #record cost
    cost_hist = []
    for i in range(num_iterations):
      #partial derivetives of m and c
        grad_m = (28*m + 12*c - 48)/3
        grad_c = (12*m + 6*c - 20)/3
        # update m and c
        m -= alpha * grad_m
        c -= alpha * grad_c
        cost = (14*m**2 + 12*m*c - 48*m + 3*c**2 - 20*c + 42)/3
        cost_hist.append(cost)
    return m, c, cost_hist
```

In [150…

```
# coodinates of three point
x_points = np.array([1, 2, 3])
y_points = np.array([1, 4, 5])
alpha = 0.1
num_iter = 300
m, c, cost_hist = model1(alpha, num_iter)
#find x and y by m and c
x_line = np.linspace(0, 4, 100)
y_line = m * x_line + c

#show cost and plot graph
print(cost_hist)
plt.figure(figsize=(8, 5))
plt.scatter(x_points, y_points, color='red', label='Data points', s=40)
plt.plot(x_line, y_line, label=f'best fit line: y = {m}x + {c}')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```

```
[1.0025424820781836, 0.6330365304374913, 0.6095514889665878, 0.59110562779
77593, 0.5735828270534521, 0.5568929321337871, 0.540995826233079, 0.525853
8446799198, 0.5114311184313465, 0.49769348232838934, 0.48460839408330497,
0.47214485719103294, 0.4602733475034147, 0.4489657432911021, 0.43819525862
76772, 0.427936379937987, 0.4181648055605483, 0.4088573881807278, 0.399992
0799984433, 0.39154788050038764, 0.38350478671313226, 0.3758437458192674,
0.3685466100242773, 0.3615960935672741, 0.35497573177379377, 0.34866984205
354373, 0.34266348675086533, 0.33694243775976673, 0.33149314281979514, 0.3
2630269341287743, 0.3213587941850757, 0.31664973382085054, 0.3121643573007
897, 0.3078920394771387, 0.30382265990451646, 0.2999465788661553, 0.296254
61453896423, 0.2927380212432193, 0.28938846872544843, 0.2861980224253576,
0.28315912468014187, 0.28026457682154887, 0.27750752212339097, 0.274881429
5590165, 0.2723800783303905, 0.26999754313195484, 0.26772818011455257, 0.2
655666135160312, 0.2635077229268982, 0.26154663116090165, 0.25967869270173
1, 0.2578994826985337, 0.25620478648414274, 0.25459058959118624, 0.2530530
6824247015, 0.2515885802930171, 0.2501936566024483, 0.24886499281709953,
0.247599441542557, 0.24639400488794885, 0.2452458273644614, 0.244152189121
0717, 0.24311049950171792, 0.24211829090839623, 0.24117321295584068, 0.240
27302690381921, 0.23941560035390333, 0.23859890219817478, 0.23782099780784
685, 0.23708004445040368, 0.23637428692451815, 0.23570205340219985, 0.2350
6175146852115, 0.23445186434944768, 0.23387094731882976, 0.233317624276085
18, 0.23279058448639964, 0.2322885794758657, 0.23181042007397204, 0.231354
973596711, 0.23092116116339412, 0.23050795514096004, 0.23011437670967658,
0.22973949354447143, 0.22938241760639735, 0.2290423030390253, 0.2287183441
647258, 0.2284097735761416, 0.22811586031834472, 0.22783590815730057, 0.22
75692539306012, 0.2273152659765619, 0.22707334263785128, 0.226842910836360
55, 0.22662342471557176, 0.22641436434757858, 0.22621523450141487, 0.22602
55634699746, 0.22584490195256754, 0.22567282199062313, 0.2255089159539238
6, 0.2253527955749893, 0.2252040910293213, 0.22506245005936876, 0.22492753
714006616, 0.2247990326840371, 0.22467663228455592, 0.22456004599441806,
0.22444899763913875, 0.2243432241627327, 0.22424247500454916, 0.2241465115
057603, 0.22405510634402503, 0.2239680429949568, 0.22388511521925616, 0.22
380612657412277, 0.22373088994795154, 0.22365922711707734, 0.2235909683235
8696, 0.22352595187317567, 0.22346402375216692, 0.2234050372626276, 0.2233
4885267489332, 0.2232953368965648, 0.2232443631572115, 0.2231958107081245
3, 0.22314956453620502, 0.22310551509159873, 0.22306355802813962, 0.223023
59395617324, 0.22298552820712794, 0.2229492706092723, 0.222914735274081,
0.22288184039281353, 0.22285050804268755, 0.2228206640023321, 0.2227922375
7592584, 0.22276516142576241, 0.22273937141272407, 0.22271480644434405, 0.
22269140833010206, 0.2226691216435602, 0.22264789359108525, 0.222627673886
77673, 0.22260841463335387, 0.22259007020868418, 0.22257259715773378, 0.22
25559540896048, 0.22254010157949247, 0.22252500207531986, 0.22251061980874
69, 0.22249692071045502, 0.2224838723294411, 0.22247144375616537, 0.222459
6055492943, 0.2224483296659893, 0.2224375893954639, 0.22242735929572652,
0.22241761513327182, 0.22240833382573064, 0.22239949338715803, 0.222391072
87593166, 0.2223830523452127, 0.22237541279562834, 0.22236813613028752, 0.
2223612051119185, 0.22235460332203635, 0.22234831512205963, 0.222342325616
21545, 0.22233662061632486, 0.2223311866081398, 0.22232601071935676, 0.222
32108068911316, 0.22231638483894756, 0.22231191204512393, 0.22230765171231
8, 0.2223035937484615, 0.22229972854088706, 0.22229604693352675, 0.2222925
402052288, 0.22228920004912092, 0.22228601855289298, 0.22228298818008105,
0.22228010175221394, 0.22227735243180766, 0.22227473370614348, 0.222272239
3718873, 0.22226986352035283, 0.22226760052351816, 0.222265445020696, 0.22
226339190585995, 0.22226143631548942, 0.22225957361710158, 0.2222577993982
687, 0.22225610945613994, 0.2222544997875057, 0.2222529665793284, 0.222251
50619966408, 0.2222501151891123, 0.2222487902525927, 0.2222475282515385,
0.2222463261964549, 0.2222451812398821, 0.2222440906695932, 0.222243051902
1993, 0.22224206247702227, 0.22224112005027052, 0.22224022238948274, 0.222
23936736824848, 0.22223855296115383, 0.22223777723899474, 0.22223703836421
294, 0.2222363345865309, 0.2222356642388045, 0.22223502573309162, 0.222234
```

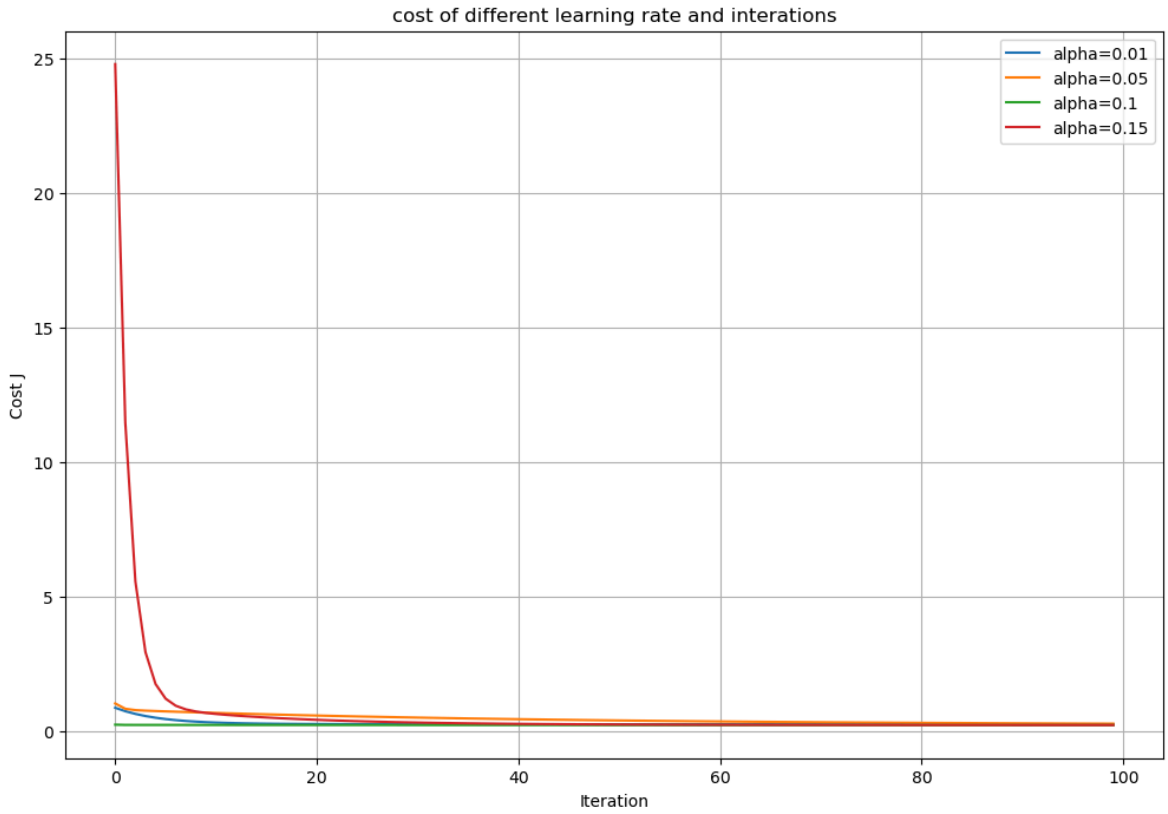
```
41755687645, 0.22223383826947676, 0.22223328649865456, 0.2222327609373522
3, 0.22223226034060417, 0.22223178352256903, 0.22223132935374204, 0.222230
89675827148, 0.22223048471140552, 0.2222300922370716, 0.22222971840555297,
0.22222936233131207, 0.22222902317085413, 0.22222870012077559, 0.222228392
41581008, 0.2222280993270592, 0.22222782016024212, 0.22222755425405202, 0.
22222730097860222, 0.2222270597339294, 0.22222682994854637, 0.222226611078
14426, 0.22222640260424717, 0.22222620403301838, 0.22222601489406202, 0.22
222583473934776, 0.22222566314211653, 0.222225499695881, 0.222225344013461
75, 0.2222251957260776, 0.2222250544824457, 0.22222491994799518, 0.2222247
9180402863, 0.2222246697469951, 0.2222245534877582, 0.22222444275092812,
0.22222433727417248, 0.22222423680765027, 0.22222414111335098, 0.222224049
9646091, 0.22222396314550016, 0.22222388045035282, 0.22222380168328945, 0.
22222372665772383, 0.22222365519592557, 0.2222235871286197, 0.222223522294
55566, 0.22222346054016384, 0.22222340171914587, 0.22222334569216903, 0.22
222329232651816, 0.22222324149577352, 0.22222319307952887, 0.2222231469630
8842, 0.22222310303721807, 0.2222230611978541, 0.22222302134589236, 0.2222
229833869278, 0.2222229472310365, 0.2222229127925838, 0.22222287998998524,
0.22222284874552636, 0.22222281898520654, 0.22222279063852568, 0.222222763
6383304, 0.22222273792066716, 0.2222227134246116, 0.22222269009213846, 0.2
2222266786796988, 0.22222264669946887, 0.22222262653648528, 0.222222607331
25646, 0.2222225890382982, 0.22222257161425793, 0.22222255501787913, 0.222
2225392098404, 0.22222252415269614, 0.22222250981077232, 0.222222496150109
8, 0.22222248313832912, 0.2222224707446235, 0.22222245893962386, 0.2222224
476953727, 0.2222224369852294, 0.22222242678382761]
```



In [151…

```python
alphas = [0.01, 0.05, 0.1,0.15]
iterations = 100

plt.figure(figsize=(12, 8))
for alpha in alphas:
    m_opt, c_opt, cost_hist = model1(alpha, iterations)
    plt.plot(cost_hist, label=f'alpha={alpha}')

plt.xlabel('Iteration')
plt.ylabel('Cost J')
plt.title("cost of different learning rate and interations")
plt.legend()
```

```
plt.grid(True)
plt.show()
```



Task A3 Machine Learning for binary classification using Logistic Regression (worth approximately 10% of the marks)

Here you will start by running some code provided to you which will create a dataset. You will use this to train a Logistic Regression classification model.

The dataset consists of a set of points (x_0, x_1) in the plane each of which is either blue or red. Your model will take as input the coordinates of a point in the plane and return 0 if the point is likely to be red, or 1 if the point is likely to be blue. In practice your model will output a float (real number) between 0 and 1 which is the probability for the point to be blue, and you will then use 0.5 as a threshold to predict the point as being blue or red.

You are guided through this task below. You will see code cells that you need to run and code cells that you need to edit/complete. You can also add new code/markdown cells as required. You may wish to add markdown cells to comment on your code or to comment on any outputs you see.

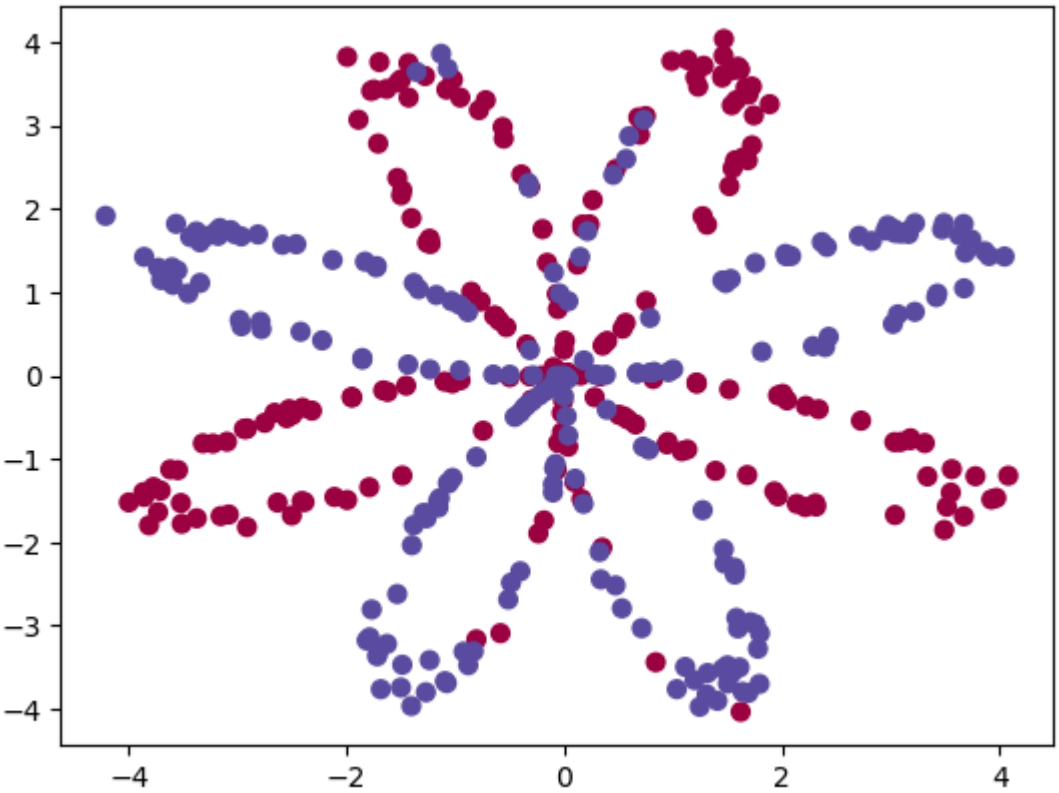
Create and visualise the dataset

Run the cell below to create a planar dataset. A visualisation of this is provided, you will see that it looks like a flower with some red points and some blue points.

In [152…

```
# Create the flower dataset

import numpy as np
import matplotlib.pyplot as plt
```

```
def load_flower_dataset():
    np.random.seed(1)

    m = 400 # number of examples
    N = int(m/2) # number of points per class
    D = 2 # dimensionality

    X = np.zeros((m,D)) # data matrix where each row is a single example
    y = np.zeros((m,1), dtype='uint8') # labels vector (0 for red, 1 for
    a = 4 # maximum ray of the flower

    for j in range(2):
        ix = range(N*j,N*(j+1))
        t = np.linspace(j*3.12,(j+1)*3.12,N) + np.random.randn(N)*0.2 # t
        r = a*np.sin(4*t) + np.random.randn(N)*0.2 # radius
        X[ix] = np.c_[r*np.sin(t), r*np.cos(t)]
        y[ix] = j

    X = X.T
    y = y.T

    return X, y

# Load the dataset:
X, y = load_flower_dataset()

# Visualize the dataset:
plt.scatter(X[0, :], X[1, :], c=y, s=40, cmap=plt.cm.Spectral);
```



You will see that you have some red points (which have label $y=0$) and some blue points (which have label $y=1$).

More precisely you have created two vectors `X` and `y` (shorthands for X_{train} and y_{train}) containing 400 training examples:

- The numpy-array `X` of dimensions (2, 400), every column of `X` is the feature vector for a single training example and contains the coordinates of the point: $\begin{pmatrix} x_0 \\ x_1\end{pmatrix}$.
- The numpy-array `y` of dimensions (1,400), every column of `y` contains the label for a single training example (0 for red, 1 for blue).
- The columns of `X` correspond to the columns of `y`, i.e. the point in the first column of `X` corresponds to the first entry in `y` and so on.

Your goal is to build a model to fit this data. In other words, you want the model to define regions of the plane as either red or blue. Red regions will correspond to points (x_0,x_1) where the model returns a value less than 0.5 and with any other parts of the plane being blue regions.

Let's start by building a logistic regression model and then measuring its performance on this problem. You will build it in numpy from scratch.

These are the steps to build your model.

1. Define the structure: $$ \hat y = \sigma\left(W \cdot x+b \right)=\sigma\left(\begin{pmatrix} w_0 & w_1\end{pmatrix}\cdot \begin{pmatrix}x_0 \\ x_1\end{pmatrix}+b\right). $$ Here the notation is the same as the one in the lecture presentation for this project. In this case, for each example, we take a linear combination $w_0x_0+w_1x_1+b$ of the input features x_0 and x_1), where w_0,w_1,b are real numbers, followed by a *sigmoid* activation function $\sigma(\lambda)=\displaystyle\frac{1}{1+\exp(-\lambda)}$. The output $\hat y$ is the model's predicted probability that the point $x=\begin{pmatrix}x_0 \\ x_1\end{pmatrix}$ is blue. Here $W = \begin{pmatrix} w_0 & w_1\end{pmatrix}$ is known as the weight matrix and b is the bias vector (which in this special case will actually be a single value, a 1×1 matrix).

2. Initialize the model's trainable parameters W and b.

3. The training loop is then as follows:

    - Implement forward propagation: compute an array whose entries are $\hat{y}^{(i)}$ for each training example, with the current values of W and b. Here $\hat{y}^{(i)}$ is the output probability for example in column i of X.
    - Compute the cost J for the current values of the parameters.
    - Implement backward propagation to get the gradients, i.e. the rate of change of the J with respect to each of w_0,w_1 and b at their current values.
    - Update the parameters using these gradients. This is known as gradient descent.

In practice, you'll build helper functions for each of these steps, and then merge them into one function called `model()`. Once you've built `model()` and used it to optimise the parameters, you can make predictions on new data.

Initialize parameters

Complete the function `initialize_parameters` started below. It takes in the dimension of the input layer `n_X` and the dimension of the output layer `n_y` and returns returns a Python dictionary `parameters`. The dictionary `parameters` contains the initialized parameters of the model: the weight matrix `W` and the bias vector `b`.

Note that `n_X = 2` in our example, each point has two input coordinates, and the dimension of the output layer is `n_y = 1`. This means that `W` will be a 1×2 matrix and `b` will be a 1×1 matrix for our example. However we would like our function to work more generally, and be in a form which can be adapted for use later on in this project, so we'd like the user to be able to specify `n_X` and `n_y` as inputs to `initialize_parameters`.

- Use: `np.random.randn(n,m) * 0.01` to randomly initialize a matrix of the required shape for `W`.
- Initialize the bias vector `b` so that all of its entries are zero.
- Return both of these in a dictionary called `parameters` with names *W* and *b* respectively.

Once you have completed the code run this cell.

In [153…

```
def initialize_parameters(n_X, n_y):
    """
    Argument:
    n_X -- integer
    n_y -- integer

    Returns:
    parameters -- python dictionary containing your parameters:
                    W -- weight matrix
                    b -- bias vector
    """

    #Initialize weight matrix and bias vector
    W = np.random.randn(n_y,n_X) * 0.01
    b = np.zeros((n_y))

    #Store W and b in a dictionary parameters
    parameters = {"W": W,
                  "b": b}

    return parameters
```

Forward Propagation

Next you will implement forward propagation for logistic regression:

$$ z = WX + b \text{ and then } \hat y = \sigma(z) $$

As above, for $\lambda\in\mathbb{R}$, $\sigma(\lambda)=\displaystyle\frac{1}{1+\exp(-\lambda)}$ (this is applied entry-wise to z in the above).

Start by implementing the sigmoid function in numpy by replacing *None* in the code cell below with appropriate code. Then run the cell.

In [154…

```
def sigmoid(z):
    return 1/(1 + np.exp(-z))
```

Complete the function `forward_propagation` started below. It calculates the output of the logistic regression model, called `y_hat`, and returns (as a tuple) `y_hat` and `z` (in the notation given above).

Once you have completed the code run this cell.

In [155…

```
def forward_propagation(X, parameters):

    # retrieve W and b from the dictionary "parameters"
    W = parameters["W"]
    b = parameters["b"]

    # compute z
    z = np.dot(W,X) + b

    # Compute y_hat
    y_hat = sigmoid(z)
    return y_hat, z
```

Compute cost

Given the predictions $\hat y^{(i)}$ on all the training examples, you can compute the cost J as the average over all training examples of the losses: $$ J = \frac{1}{m} \sum\limits_{i = 0}^{m} L(\hat y^{(i)}, y^{(i)}) . $$ Here we are using the binary cross-entropy loss: $$ L(\hat y^{(i)}, y^{(i)}) = -\large\left(\small y^{(i)}\log\left(\hat y^{(i)}\right) + (1-y^{(i)})\log\left(1- \hat y^{(i)}\right) \large \right). $$

Recall $y^{(i)}$ is the ith component in y, i.e. the label for example i and $\hat{y}^{(i)}$ is the models current prediction of this label, the ith component in $\hat{y}$.

Complete the function `compute_cost()`, started below, to compute the value of the cost J:

1. First compute the vector of losses element-wise using the numpy arrays given as inputs `y_hat` and `y`.
2. Sum the losses and divide by the number of training examples `m`. Call this value `cost`.
3. Cast `cost` as type `float` using `float()` and return it.

Once you have completed the code run this cell.

In [156…

```
def compute_cost(y_hat, y):
    # retrieve number of training examples from the shape of y
    m = np.shape(y)[1]

    # compute the vector of losses by computing the cross-entropy loss el
    L = -(y*np.log(y_hat) + (1-y)*np.log(1-y_hat))

    # compute the total cost by averaging the loss over all training exam
    cost = (1/m)*np.sum(L)
    return float(cost)
```

Backpropagation

You can now implement backward propagation. Note that you will complete the code cell below this, much of the code is already given to you but you need to stick with the notation established.

NOTATION: in general for a generic numpy array `M` and a function F with the entries of `M` as inputs, denote by `dM` the numpy array representing the gradient of F with respect to `M`. More precisely `dM` contains the partial derivatives of F with respect to the entries of `M`. For example if $$M = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} \\ a_{1,0} & a_{1,1} & a_{1,2}\end{pmatrix}$$ and F is a function of $a_{0,0},a_{0,1},a_{0,2},a_{1,0},a_{1,1},a_{1,2}$ then $$dM = \begin{pmatrix} \frac{\partial F}{\partial a_{0,0}} & \frac{\partial F}{\partial a_{0,1}} & \frac{\partial F}{\partial a_{0,2}} \\ \frac{\partial F}{\partial a_{1,0}} & \frac{\partial F}{\partial a_{1,1}} & \frac{\partial F}{\partial a_{1,2}}\end{pmatrix}.$$

Throught this project F will be the cost function J.

Complete the function `backward_propagation()` started below. It takes in a dictionary `parameters` containing the current parameters, `y_hat`, `X` and `y` and it returns a dictionary containing the gradients `dW` and `db` of the cost function with respect to the trainable parameters `W` and `b`.

You are given the key code for calculating `dW` and `db`, it is, in the notation already established.

`dz = y_hat - y dW = np.dot(dz, X.T)/m db = np.sum(dz, axis=1, keepdims=True)/m`

You will see these lines of code in the cell below.

Once you have completed the code run this cell.

In [157…

```
def backward_propagation(parameters, y_hat, X, y):
    """
    Implement the backward propagation.

    Arguments:
    parameters -- python dictionary containing W and b to use
    y_hat - from output of forward prop
```

```
    X -- input data
    y -- labels vector

    Returns:
    grads -- python dictionary containing the gradients
    """
    # Retrieve the number of training examples from the shape of y
    m = np.shape(y)[1]

    # retrieve W from the dictionary "parameters".
    W = parameters['W']

    # Backward propagation: calculate dW, and db.
    dz = y_hat - y
    dW = np.dot(dz, X.T)/m
    db = np.sum(dz, axis=1, keepdims=True)/m

    grads = {"dW": dW,
             "db": db}

    return grads
```

Update parameters

Next you will define a function to update parameters using gradient descent. In a similar way to that seen in task A2,

1. W will be updated to

$$W - dW\times \text{learning rate}.$$

and

2. b will be updated to$$b - db\times\text{learning rate}.$$

Complete the function `update_parameters()` started below. It takes in the dictionary `parameters` containing the current parameters `W` and `b`, the dictionary `grads` containing the current gradient `dW`, `dB` and the learning rate (which defaults to the value 1.2). **Once you have completed the code run this cell.**

In [158…

```
def update_parameters(parameters, grads, learning_rate = 1.2):

    # Retrieve a copy of each parameter from the dictionary "parameters".
    W = copy.deepcopy(parameters["W"])
    b = parameters["b"]

    # Retrieve each gradient from the dictionary "grads"
    dW = grads["dW"]
    db = grads["db"]

    # Update each parameter
    new_W = W - dW*learning_rate
    new_b = b - db*learning_rate

    # Store the new parameters in the dictionary parameters
    parameters = {"W": new_W,
```

```
                    "b": new_b}


    return parameters
```

Model

Now you will put everything together into a function `model` where you will apply `update_parameters` iteratively to update the parameters a number of times.

Complete the function `model` started below. It takes as inputs the training data `X`, `y`, a positive integer `num_iterations` which is the number of times the parameters will be updated (defaulting to $10,000$) and a boolean `print_cost` which, when set to `True`, will mean the function prints out some costs as the values of the parameters are updated). `model` return a dictionary containing the optimised parameters at the end of this process. **Once you have completed the code run this cell.**

In [159…

```
def model(X, y, num_iterations = 10000, print_cost=True):

    # retrieve n_x and n_y from X and y
    n_X = np.shape(X)[0]
    n_y = np.shape(y)[0]

    # Initialize parameters
    parameters = initialize_parameters(n_X, n_y)

    # Training loop (gradient descent)
    for i in range(0, num_iterations):

        # Forward propagation
        y_hat, z = forward_propagation(X, parameters)

        # Compute the cost
        cost = compute_cost(y_hat, y)

        # Backpropagation
        grads = backward_propagation(parameters, y_hat, X, y)

        # Update parameters (use learning_rate = 1.2)
        parameters = update_parameters(parameters, grads, learning_rate =

        # Print the cost every 1000 iterations
        if print_cost and i % 1000 == 0:
            print ("Cost after iteration %i: %f" %(i, cost))

    learned_parameters = parameters
    return learned_parameters
```

Predict on new data

Below you are given a complete function `predict` that uses the learned parameters (output of `model`) and the `forward_propagation` function to predict a class for each example in the input matrix `X`. **Run the code cell below.**

In [160…

```
def predict(learned_parameters, X):
    """
    Arguments:
    parameters -- python dictionary containing the learned parameters giv
    X -- numpy array input data

    Returns
    predictions -- vector of predictions for each column of X (red: 0 / b
    """

    # Compute predicted probabilities using the learned parameters
    y_hat, z = forward_propagation(X, learned_parameters)

    # Classify as 0 or 1 using 0.5 as a threshold
    predictions = (y_hat > 0.5)
    return predictions
```

Plot the results and calculate accuracy

You are given the complete function `plot_decision_boundary` below which plots the decision boundary of the trained model along with the original data. It takes as arguments the `learned_parameters` (as output by the `model` function), the model, the training examples `X` and the true labels `y`. **Run the code cell below.**

In [161…

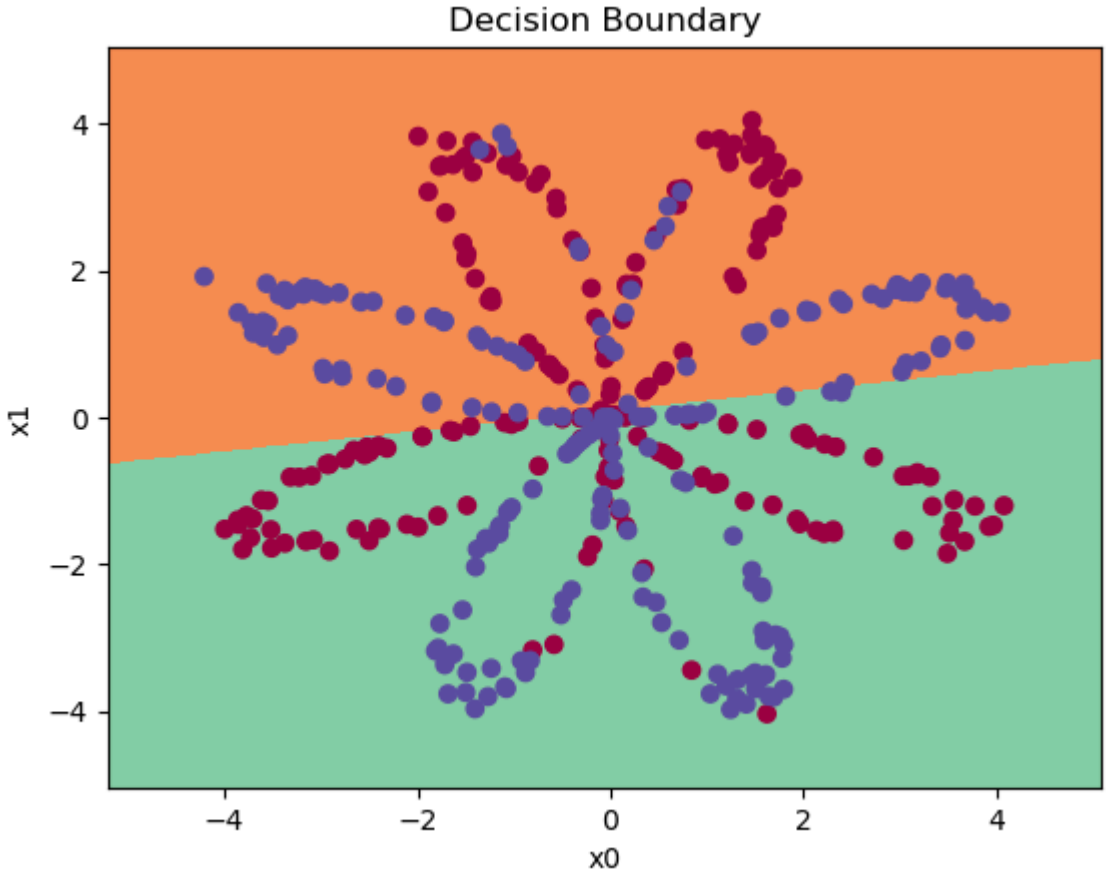
```
def plot_decision_boundary(learned_parameters, X, y):

    # Set min and max values and give it some padding
    x_min, x_max = X[0, :].min() - 1, X[0, :].max() + 1
    y_min, y_max = X[1, :].min() - 1, X[1, :].max() + 1
    h = 0.01

    # Generate a grid of points with distance h between them
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_m

    # Predict the function value for the whole grid
    Z = predict(learned_parameters,np.c_[xx.ravel(), yy.ravel()].T)
    Z = Z.reshape(xx.shape)

    # Plot the contour and training examples
    plt.contourf(xx, yy, Z, cmap=plt.cm.Spectral)
    plt.ylabel('x1')
    plt.xlabel('x0')
    plt.scatter(X[0, :], X[1, :], c=y, cmap=plt.cm.Spectral)
    plt.title("Decision Boundary")
    plt.show()
```

Now you can train your model using the training examples, and plot the decision boundary for the learned parameters, alongside the orginal data. **Run the code cell below**.

In [162…

```
# Train your model
learned_parameters = model(X, y)

# Plot the decision boundary
plot_decision_boundary(learned_parameters, X, y)
predictions = predict(learned_parameters, X)
```

```
Cost after iteration 0: 0.694565
Cost after iteration 1000: 0.673146
Cost after iteration 2000: 0.673146
Cost after iteration 3000: 0.673146
Cost after iteration 4000: 0.673146
Cost after iteration 5000: 0.673146
Cost after iteration 6000: 0.673146
Cost after iteration 7000: 0.673146
Cost after iteration 8000: 0.673146
Cost after iteration 9000: 0.673146
```



As you should see logistic regression doesn't do a good job in predicting the labels for this dataset. This is because the dataset is not linearly separable, and therefore a simple logistic regression performs poorly on this data.

Complete the code cell below with a formula for the accuracy of the model. This should be the precentage of points for which the model gives the correct classification. **Then run the cell.**

In [163…

```
# Print accuracy
def calculate_accuracy(learned_parameters):

    #Retrieve predicted values
    predictions = predict(learned_parameters, X)

    #Calculate the accuracy
    num_incorrect = np.sum(np.bitwise_xor(predictions,y))
    total = len(y[0])
    accuracy = 100*(total - num_incorrect)/total

    return accuracy
```

```
#Print out the accuracy
print ('Accuracy of logistic regression:' + str(round(calculate_accuracy(
       + '% ' + "(percentage of correctly labelled datapoints)")
```

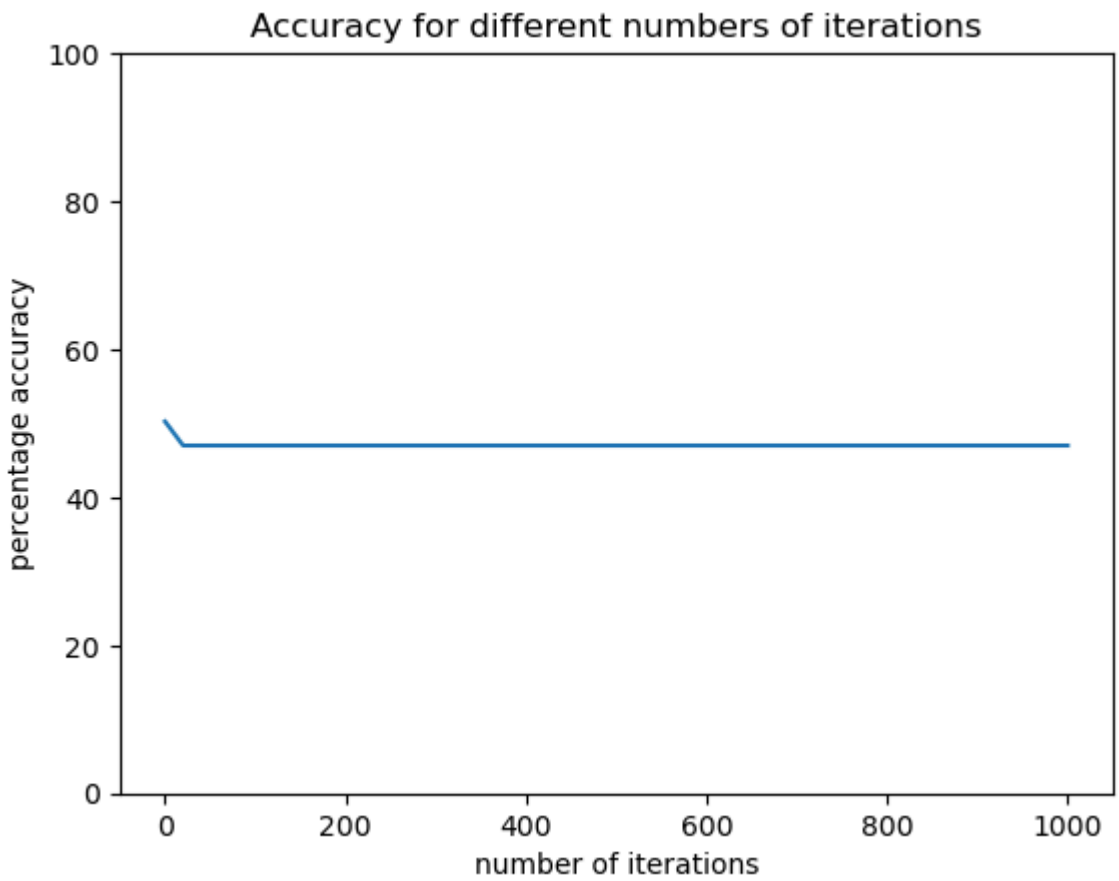
```
Accuracy of logistic regression:47.0% (percentage of correctly labelled da
tapoints)
```

Number of Iterations Required for A3

The code below runs the model many times with varying numbers of iterations. This is to gain some insight into how quickly the model converges on a solution.

In [164…

```
#Run for different values of the model
x_range = np.linspace(0,1000,51,dtype=int)
y_range = np.array([calculate_accuracy(model(X, y, num_iterations = i, pr

plt.ylabel('percentage accuracy')
plt.xlabel('number of iterations')
plt.ylim(bottom=0, top=100)
plt.plot(x_range,y_range)
plt.title("Accuracy for different numbers of iterations")
plt.show()
```



A4 - Build a hidden layer Neural Network (worth approximately 20% of the marks)

You are now going to build a full Neural Network with one hidden layer. This will allow to model to learn more complex decision boundaries and not just linear ones. This will result in an improved model compared to that you obtained in task A3.

In practice you are going to repeat all the steps of task A3 but this time with an extra layer to the network. Set the hidden layer to have 4 neurons (in the code this dimension is denoted `n_h`). You may find it helpful to refer to the project lecture when you are completing this section.

The forward propagation has the form `linear` -> `tanh` -> `linear` -> `sigmoid`. So this will take the form below where W and T are appropriately sized matrices containing weights and b and c are appropriately sized column vectors for the biases (some of the dimensions of these objects will now depend on `n_h`). The input matrix X is the same as in task A3, z, a, v give intermediate output with $\hat y$ the final output.

$$ z = W\cdot X + b \\ a = \tanh(z) \\ v = T \cdot a + c \\ \hat y = \sigma(v) $$

The code for the new backward propogation function, in this notation, is as follows and you can cut and paste this into your code as appropriate.

```
dv = y_hat - y dT = np.dot(dv, a.T)/m dc = np.sum(dv, axis=1, keepdims=True)/m dz = np.dot(T.T, dv) * (1 - np.power(a, 2)) dW = np.dot(dz, X.T)/m db = np.sum(dz, axis=1, keepdims=True)/m
```

The cost is the same as for logistic regression, so there is no need to implement it again.

Neural Network Model

Below gives a basic structure for your final model, you need to build all the helper functions from scratch this time.

Specifically, using an approach similar to that seen in A3, you should

- Complete the function below `model_nn`, including all the necessary helper functions.
- Include a function `predict` which takes in the dictionary of paremeters and input data 'X' and returns predictions of the labels for each example.
- Produce a visualisation of the regions of the plane predicted to be red/blue by your model and the original data.
- Calculate and print the accuracy of the model (the percentage of points for which the model predicts the correct label).
- Make some comments, in a markdown cell, which compare the model developed in A3 to the model developed in A4.

You may use code from Task A3 here but be careful to rename functions and variables as appropriate to avoid clashes with any values established for this notebook in Task A3.

Edit the code cell below and insert further code and markdown cells in which to answer this task

The functions below are very similar to those in part 4 and perform the same functions. I have used the naming convention function_name4 to distinguish them from the functions in A3 (function_name).

In [165…

```
def initialize_parameters4(n_X, n_y, n_h):

    #Initialize weight matrix nd bias vector
    W = np.random.randn(n_h,n_X) * 0.01
    b = np.zeros((n_h,1))
    T = np.random.randn(n_y,n_h) * 0.01
    c = np.zeros((n_y, 1))

    #Store W and b in a dictionary parameters
    parameters = {"W": W,
                  "b": b,
                  "T": T,
                  "c": c}
    return parameters
```

In [166…

```
def forward_propagation4(X, parameters):
    # Retrieve W, b, and t from the dictionary "parameters"
    W = parameters["W"]
    b = parameters["b"]
    T = parameters["T"]
    c = parameters["c"]

    # Compute z, a
    z = np.dot(W, X) + b
    a = np.tanh(z)

    # Compute v
    v = (T @ a) + c

    # Compute y_hat
    y_hat = sigmoid(v)
    return y_hat, a
```

In [167…

```
def backward_propagation4(parameters, y_hat, X, y, a):
    """
    Implement the backward propagation.

    Arguments:
    parameters -- python dictionary containing W and b to use
    y_hat - from output of forward prop
    X -- input data
    y -- labels vector

    Returns:
    grads -- python dictionary containing the gradients
    """
    # Retrieve the number of training examples from the shape of y
    m = np.shape(y)[1]

    # retrieve W,T from the dictionary "parameters".
```

```
    W = parameters['W']
    T = parameters['T']

    # Backward propagation: calculate dW, db, dT and dc.
    dv = y_hat - y
    dT = np.dot(dv, a.T)/m
    dc = np.sum(dv, axis=1, keepdims=True)/m
    dz = np.dot(T.T, dv) * (1 - np.power(a, 2))
    dW = np.dot(dz, X.T)/m
    db = np.sum(dz, axis=1, keepdims=True)/m

    # Return all partial derivatives as a dictionary
    grads = {"dW": dW,
             "db": db,
             "dT": dT,
             "dc": dc}

    return grads
```

In [168…

```
def update_parameters4(parameters, grads, learning_rate = 1.2):

    # Retrieve a copy of each parameter from the dictionary "parameters".
    W = copy.deepcopy(parameters["W"])
    b = parameters["b"]
    T = copy.deepcopy(parameters["T"])
    c = parameters["c"]

    # Retrieve each gradient from the dictionary "grads"
    dW = grads["dW"]
    db = grads["db"]
    dT = grads["dT"]
    dc = grads["dc"]

    # Update each parameter
    new_W = W - dW*learning_rate
    new_b = b - db*learning_rate
    new_T = T - dT*learning_rate
    new_c = c - dc*learning_rate

    # Store the new parameters in the dictionary parameters
    new_parameters = {"W": new_W,
                      "b": new_b,
                      "T": new_T,
                      "c": new_c}

    return new_parameters
```

In [169…

```
def model_nn(X, y, n_h = 4, num_iterations = 10000, print_cost=False):
    """
    Arguments:
    X -- numpy array, input data
    y -- numpy array, true labels
    n_h -- integer, number of neurons in the hidden layer
    num_iterations -- number of iterations in the gradient descent loop
    print_cost -- if True, print the cost every 1000 iterations

    Returns:
    parameters -- optimized parameters learnt by the model.
    """
```

```
    # retrieve n_x and n_y
    n_X = np.shape(X)[0]
    n_y = np.shape(y)[0]

    # Initialize parameters
    parameters4 = initialize_parameters4(n_X, n_y, n_h)

    # Training loop
    for i in range(0, num_iterations):

        # Forward propagation
        y_hat, a = forward_propagation4(X, parameters4)

        # Compute cost
        cost = compute_cost(y_hat, y)

        # Backpropagation
        grads = backward_propagation4(parameters4, y_hat, X, y, a)

        # Update parameters
        parameters4 = update_parameters4(parameters4, grads, learning_rat

        # Print the cost every 1000 iterations
        if print_cost and i % 1000 == 0:
            print ("Cost after iteration %i: %f" %(i, cost))

    learned_parameters4 = parameters4
    return learned_parameters4
```

In [170…

```
def predict4(learned_parameters, X):

    # Compute predicted probabilities using the learned parameters
    y_hat, a = forward_propagation4(X, learned_parameters)

    # Classify as 0 or 1 using 0.5 as a threshold
    predictions = (y_hat > 0.5)
    return predictions
```

In [171…

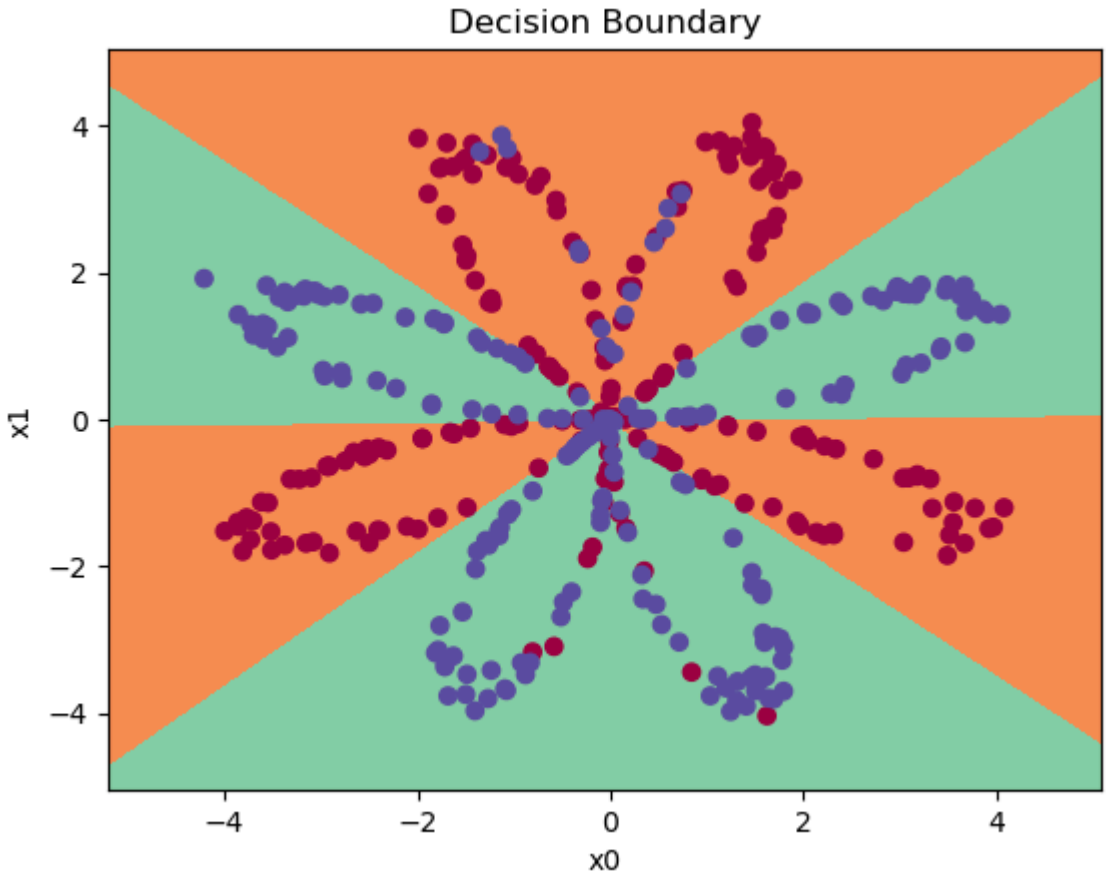
```
# Function to plot decision boundary
def plot_decision_boundary4(learned_parameters, X, y):

    # Set min and max values and give it some padding
    x_min, x_max = X[0, :].min() - 1, X[0, :].max() + 1
    y_min, y_max = X[1, :].min() - 1, X[1, :].max() + 1
    h = 0.01

    # Generate a grid of points with distance h between them
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_m

    # Predict the function value for the whole grid
    Z = predict4(learned_parameters,np.c_[xx.ravel(), yy.ravel()].T)
    Z = Z.reshape(xx.shape)

    # Plot the contour and training examples
    plt.contourf(xx, yy, Z, cmap=plt.cm.Spectral)
    plt.ylabel('x1')
    plt.xlabel('x0')
    plt.scatter(X[0, :], X[1, :], c=y, cmap=plt.cm.Spectral)
```

```
    plt.title("Decision Boundary")
    plt.show()
```

In [172…

```
# Train your model
learned_parameters4 = model_nn(X, y, n_h = 4, num_iterations = 10000, pri

# Plot the decision boundary
plot_decision_boundary4(learned_parameters4, X, y)

predictions = predict4(learned_parameters4, X)
```

```
Cost after iteration 0: 0.693162
Cost after iteration 1000: 0.256675
Cost after iteration 2000: 0.238684
Cost after iteration 3000: 0.230430
Cost after iteration 4000: 0.225276
Cost after iteration 5000: 0.221659
Cost after iteration 6000: 0.218951
Cost after iteration 7000: 0.220518
Cost after iteration 8000: 0.219353
Cost after iteration 9000: 0.218481
```



In [173…

```
# Print accuracy
def calculate_accuracy4(learned_parameters):
    #Retrieve predicted values
    predictions = predict4(learned_parameters, X)

    #Calculate the accuracy
    num_incorrect = np.sum(np.bitwise_xor(predictions,y))
    total = len(y[0])
    accuracy = 100*(total - num_incorrect)/total

    return accuracy
```

```
#Print out the accuracy
print ('Accuracy of logistic regression: ' + str(round(calculate_accuracy
       + '% ' + "(percentage of correctly labelled datapoints)")
```

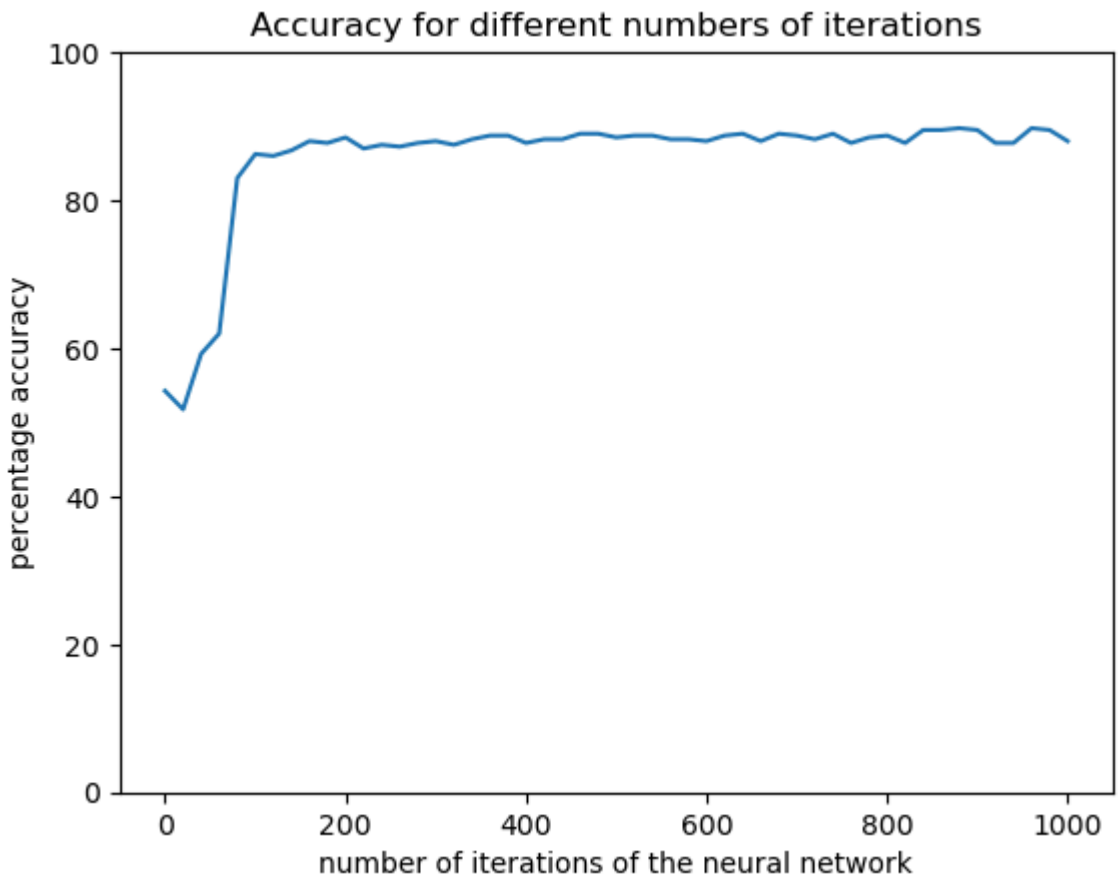
```
Accuracy of logistic regression: 90.5% (percentage of correctly labelled d
atapoints)
```

Number of Iterations Required for A4

The code below runs the model many times with varying numbers of iterations. This is to gain some insight into how quickly the model converges on a solution. Note that it converges more slowly that in A3, but to a higher accuracy.

In [174…

```
# Run for different values of the model
x_range = np.linspace(0,1000,51,dtype=int)
y_range = np.array([calculate_accuracy4(model_nn(X, y, num_iterations = i

# Plot the results
plt.ylabel('percentage accuracy')
plt.xlabel('number of iterations of the neural network')
plt.ylim(bottom=0, top=100)
plt.plot(x_range,y_range)
plt.title("Accuracy for different numbers of iterations")
plt.show()
```



Comparison of this model to the model in A3

- This neural network model has more weights, and is therefore able to give a more complicated output (not just a linear split of the data).
- This model therefore has a much higher accuracy (90% compared to 50%).
- This model appears to have the right level of complexity, such that each petal has been distinguished as expected, but individual outlier points in the training data aren't affecting the model (no overfitting).
- This model continues improving (reducing the cost) after 1000 iterations, whereas the model in A3 has stopped learning by this point.

Section B (worth approximately 40%)

Remember you are free to use sklearn in this section.

Task B1

- How can we further improve neural networks? You may wish to talk about underfitting, overfitting, regularisation and use examples to illustrate you ideas.

Task B2

- What are some other machine learning models (e.g. decision trees). Give some examples. What are improvements to these (e.g random forest, xgboost in the case of decision trees)?

You may wish to use datasets from https://archive.ics.uci.edu/ in this section.

Vanishing Gradient Problem

The code below simulates the vanishing gradient problem for neural networks. It uses Sklearn to create 3 neural network models with different activation functions (sigmoid, tanh, and RelU), where the number of hidden layers can be varied.

These models are then trained on the data from A3 and A4, and their accuracy is then plotted.

Function to train the 3 models

In [175…

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split

# Train test split
X_train, X_test, y_train, y_test = train_test_split(X.T, y.ravel(), test_

def train_3_models(num_hidden_layers, node_size, num_iter=500):
```

```
    # Create the 3 models - all multilayer perceptron neural networks wit
    relu_model = MLPClassifier(hidden_layer_sizes=(node_size,) * num_hidd
    tanh_model = MLPClassifier(hidden_layer_sizes=(node_size,) * num_hidd
    sigmoid_model = MLPClassifier(hidden_layer_sizes=(node_size,) * num_h

    # Fit the data to all 3 models
    relu_model.fit(X_train, y_train)
    tanh_model.fit(X_train, y_train)
    sigmoid_model.fit(X_train, y_train)

    # Use the mdoels to make predictions on the test data
    relu_prediction = relu_model.predict(X_test)
    tanh_prediction = tanh_model.predict(X_test)
    sigmoid_prediction = sigmoid_model.predict(X_test)

    return [relu_prediction, tanh_prediction, sigmoid_prediction]
```

Function to calculate percentage of points correctly predicted by the model

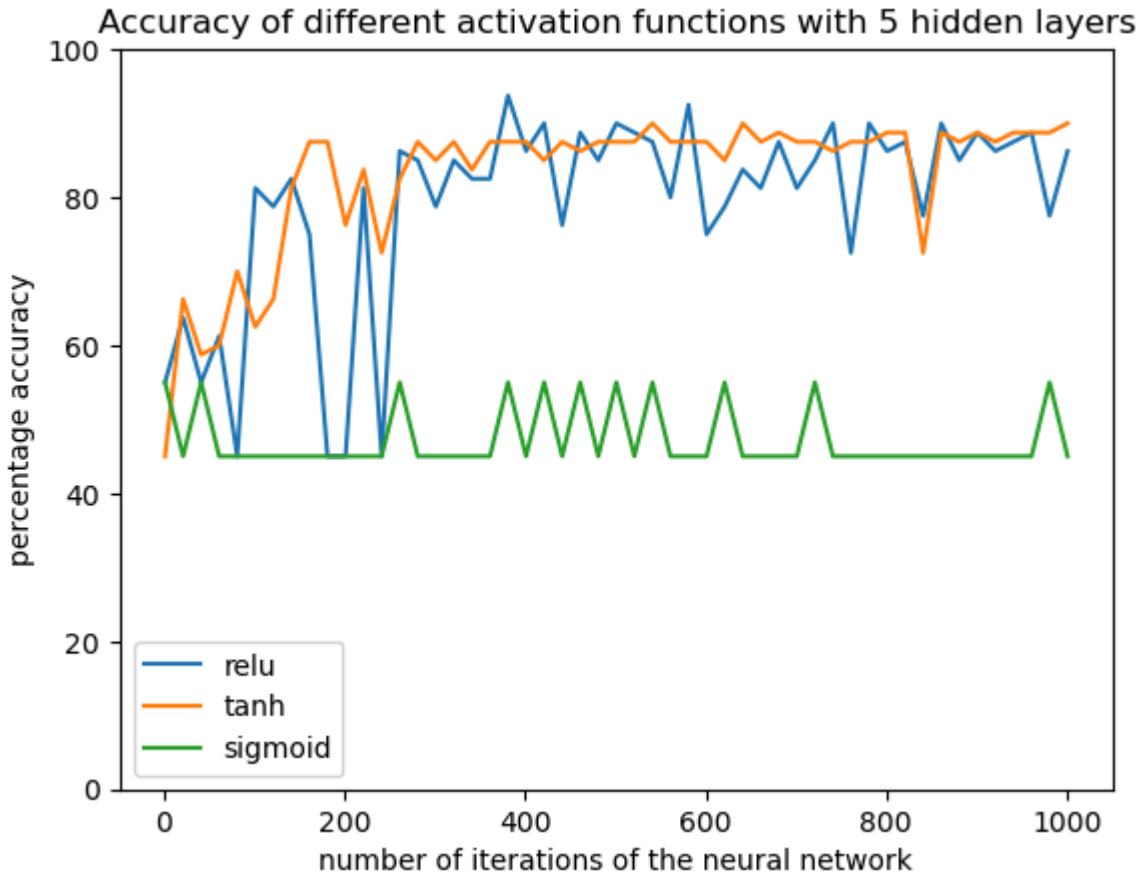
In [176…

```
def calculate_accuracy_grad_descent(prediction):
    num_incorrect = np.sum(np.bitwise_xor(prediction,y_test))
    total = len(y_test)
    accuracy = 100*(total - num_incorrect)/total
    return accuracy
```

Plotting a graph showing the accuracy of each model with 5 hidden layers

In [177…

```
# Ignore warnings (caused by using a small number of interations and the
from warnings import simplefilter
from sklearn.exceptions import ConvergenceWarning
simplefilter("ignore", category=ConvergenceWarning)

# Number of hidden layers + number of nodes in each layer
num_hidden_layers = 5
node_size = 5
max_iterations = 1000
num_points = 50

# Create x range and y range, which calculates the accuracy of each model
x_range = np.linspace(1,max_iterations+1,num_points+1,dtype=int)
y_ranges = np.array([list(map(calculate_accuracy_grad_descent, train_3_mo

# Create labels
plt.ylabel('percentage accuracy')
plt.xlabel('number of iterations of the neural network')
plt.ylim(bottom=0, top=100)

text = "Accuracy of different activation functions with " + str(num_hidde
plt.title(text)

# Plot all 3 functions on the same graph
plt.plot(x_range,y_ranges[:, 0],label='relu')
plt.plot(x_range,y_ranges[:, 1],label='tanh')
plt.plot(x_range,y_ranges[:, 2],label='sigmoid')
plt.legend()
plt.show()
```



The vanishing gradient problem can be clearly seen in the inability of the sigmoid function to accurately predict the data. When running this code with only one hidden layers (as shown in our presentation), the sigmoid function does just as well as the other two.

In [ ]:

Overfitting, Underfitting, and Regularisation

Underfitting occurs when a model is too simplistic and doesn't capture the correct correlation of the data poitns

Overfitting is the opposite, where a model is too complex and focuses on memorising data instead of investigating a trent

Both overfitting and underfitting lead to inacurate models which can't be generalised to predicted data points

The code below investigates regularisation: a technique used to reduce overfitting. Specifically, it uses:

- Lasso regression (L1),
- Ridge regression (L2),
- Elastic net regression (a combination of L1 and L2).

We use Skikit-learn to generate a dataset of 100 samples, and a linear regression model.

The following libaries are needed:

In [178…

```
#Importing Libraries:
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.linear_model import Lasso
from sklearn.linear_model import Ridge
from sklearn.linear_model import ElasticNet
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error
```

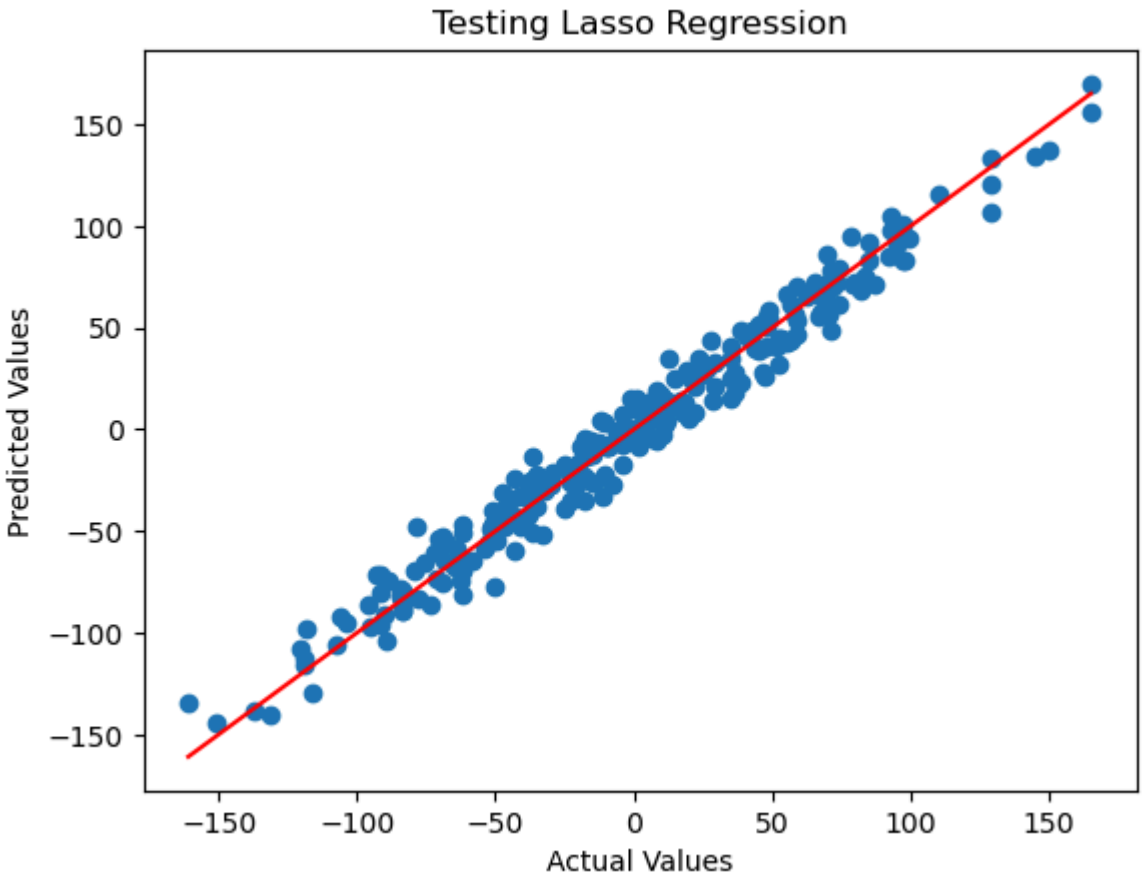
Lasso Regression

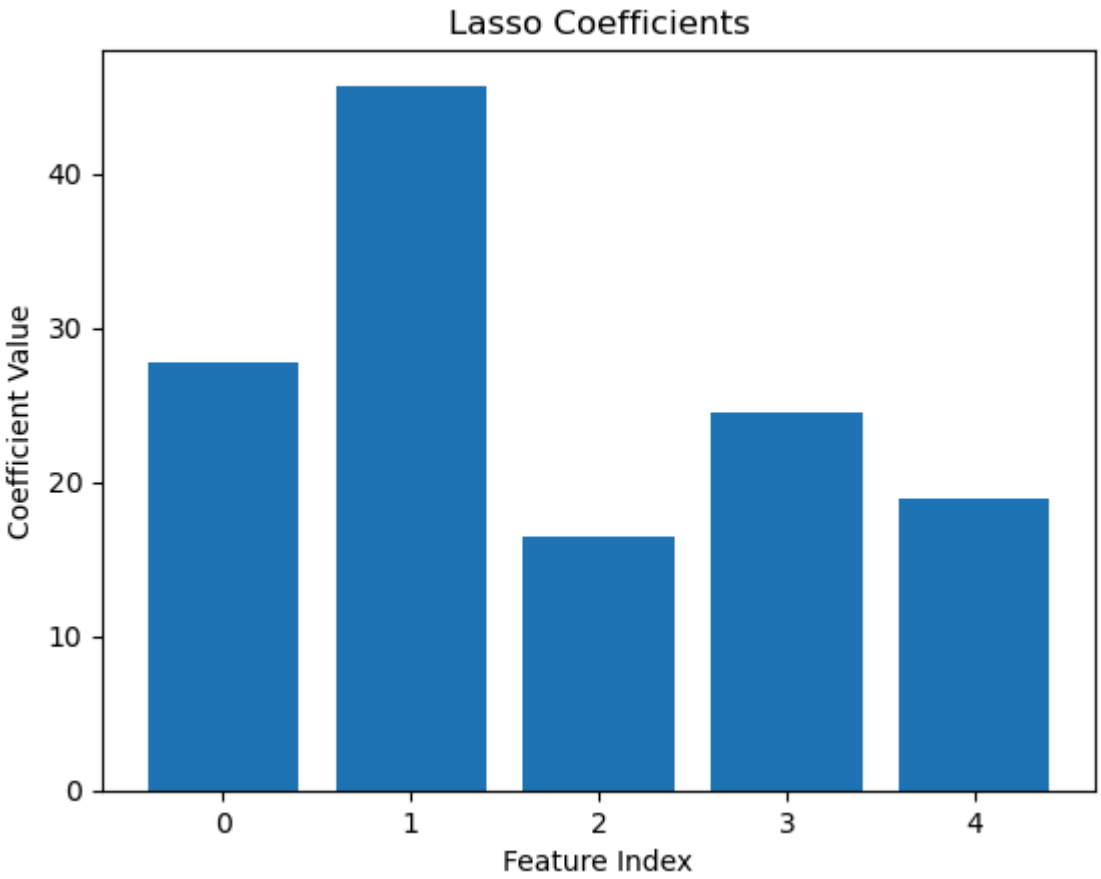
Lasso regression introduces a penalty function to the model to prevent it from memorising data. Using L1 regularisation, it penalises and reduces large coefficients to prevent overfitting. An example is shown below:

In [179…

```
#Lasso Regularisation

#Generating a data set with "noise"
X, y = make_regression(n_samples=1000, n_features=5, noise=10, random_sta
#Test/Train split (75%/25%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,

#Creates Lasso Regulation Model
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print("Coefficients:", lasso.coef_)



#Plotting the graphs
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Testing Lasso Regression")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='r
plt.show()

plt.bar(range(len(lasso.coef_)), lasso.coef_)
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.title("Lasso Coefficients")
plt.show()
```

```
Mean Squared Error: 106.71830775277488
Coefficients: [27.77287707 45.71886486 16.51247725 24.48948791 18.9791759
2]
```





As seen above, the model captures the general relation of the dataset instead of memorising points (thus not overfitting), even with a moderate amount of noise

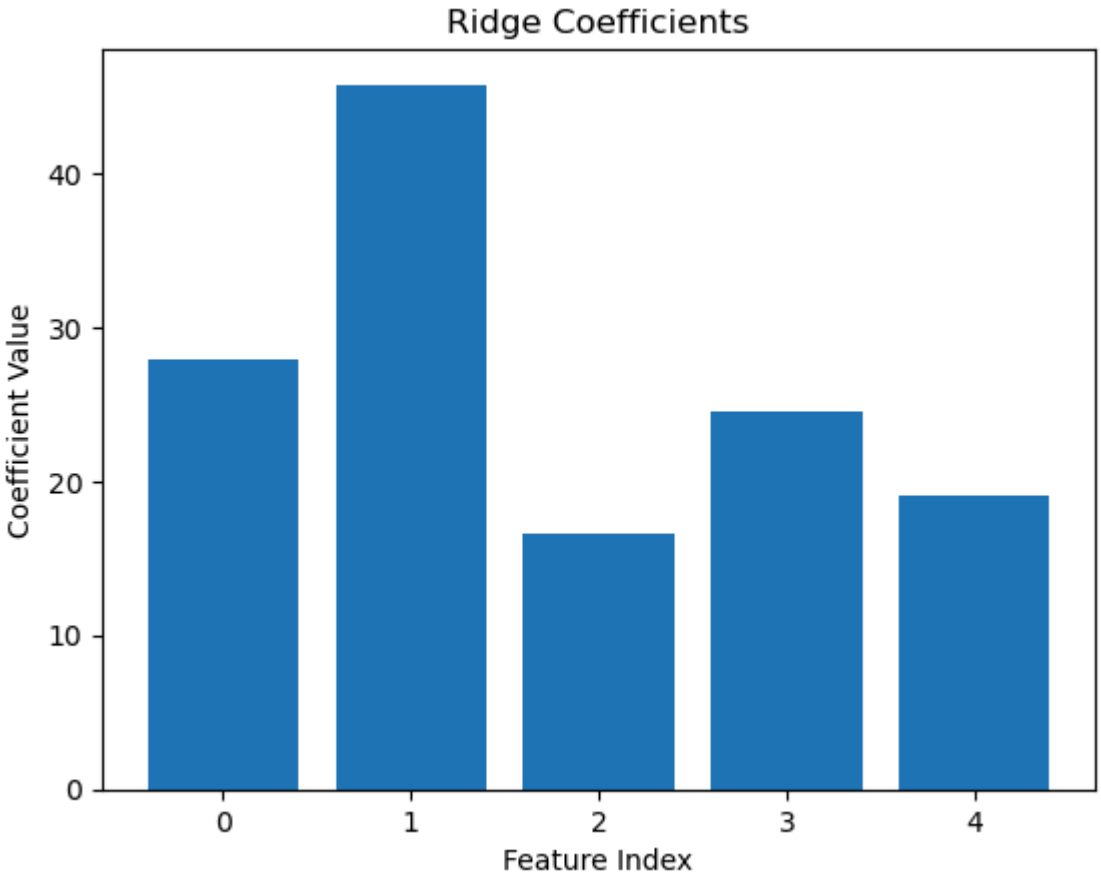
Ridge Regression

Ridge regressions is very similar to Lasso, except it uses L2 instead of L1, meaning penalised coefficients can never equal 0, but will be reduced close to 0.

In [180…

```
#Ridge Regression

#Generating a data set with noise
X, y = make_regression(n_samples=1000, n_features=5, noise=10, random_sta
#Test/Train split (75%/25%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,

#Create Ridge |Regression Model
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse) # low MSE = less error
print("Coefficients:", ridge.coef_)

#Plotting the graphs
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Testing Ridge Regression")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='r
plt.show()

plt.bar(range(len(ridge.coef_)), ridge.coef_)
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.title("Ridge Coefficients")
plt.show()
```

```
Mean Squared Error: 106.40294539102979
Coefficients: [27.84530735 45.75008987 16.58553263 24.55558597 19.0543869
8]
```



As seen above, Lasso and Ridge give very close coefficients to eachother and therfore a very similar graph

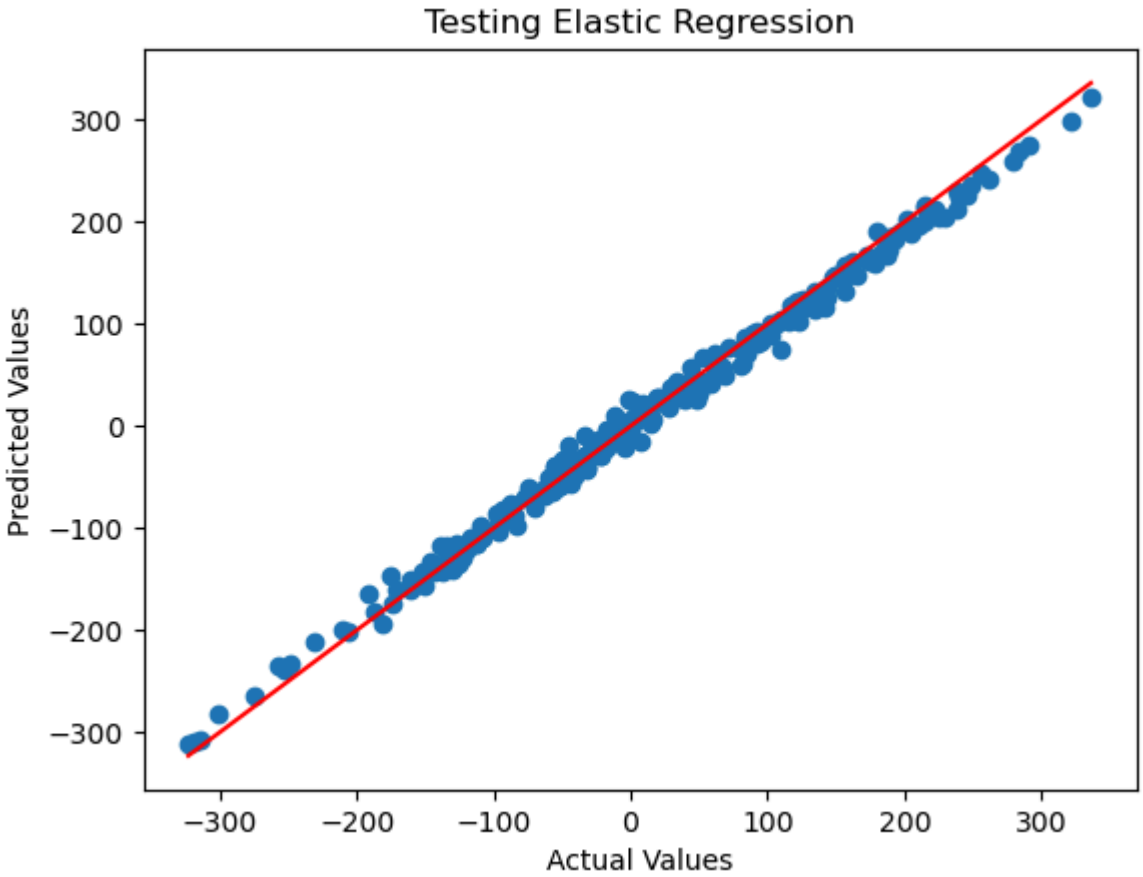
Elastic Net Regression

Elastic net regression is ...

In [181…

```
#Elastic Net Regression
#Making a dataset with some noise
X, y = make_regression(n_samples=1000, n_features=10, noise=10, random_st
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,

model = ElasticNet(alpha=0.1, l1_ratio=0.5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
print("Coefficients:", model.coef_)

#Plotting the graphs
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Testing Elastic Regression")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='r
plt.show()
```

```
Mean Squared Error: 132.38939525913761
Coefficients: [31.59908759 29.97790627 27.66309441 70.83563433  6.66348031
9.34101356
 72.1213741   8.92896938  4.35680654 56.29586598]
```



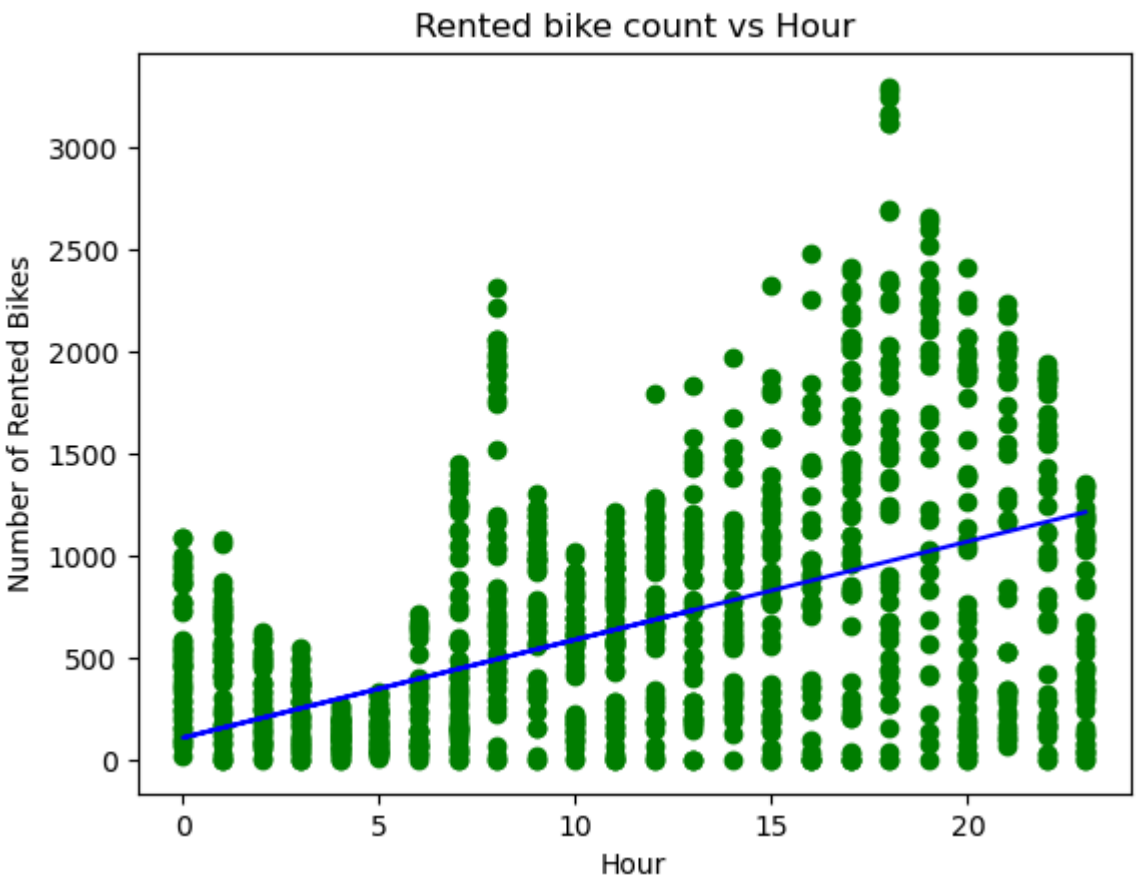
Below shows a following example of using an Elastic Regression model on the Seoul Bike Data set (used in A1)

In [182…

```
#Build Elastic Net Regression:
class ElasticRegression:
#initialises learning rate, iterations and penalties
    def __init__(self, learning_rate, iterations, l1_penalty, l2_penalty)
        self.learning_rate = learning_rate
        self.iterations = iterations
        self.l1_penalty = l1_penalty
        self.l2_penalty = l2_penalty
#initialises weights
#NOTE: model is of form Y = a*X + b
    def fit(self, X, Y):
        self.m, self.n = X.shape
        self.a = np.zeros(self.n)
        self.b = 0
#storing the data
        self.X = X
        self.Y = Y
#training loop
        for i in range(self.iterations):
            self.new_weights()
        return self
#computes gradients and adds penalties
    def new_weights(self):
#Makes predictions about outcome value
        Y_pred = self.predict(self.X)
        da = np.zeros(self.n)
        for j in range(self.n):
#Adds L1 Penalty
            l1_grad = self.l1_penalty if self.a[j] > 0 else -self.l1_pena
            da[j] = (
                -2 * (self.X[:, j]).dot(self.Y - Y_pred) +
                l1_grad + 2 * self.l2_penalty * self.a[j]
            ) / self.m
        db = -2 * np.sum(self.Y - Y_pred) / self.m
#Updating Perameters
        self.a -= self.learning_rate * da
        self.b -= self.learning_rate * db
        return self
#predicting values
    def predict(self, X):
        return X.dot(self.a) + self.b


#Reading the dataset
df = pd.read_csv("Seoul Bike Data.csv")
df = df.iloc[:, :2]
X = df.iloc[:, 1].values.reshape(-1, 1)
Y = df.iloc[:, 0].values
#Test/Train split (75%/25%)
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.25, random_state=0)

#Setting the penalty strengths
model = ElasticRegression(
    iterations=1000,
    learning_rate=0.001, #learning rate has to be small to avoid error
```

```
    l1_penalty=1,
    l2_penalty=1
)
model.fit(X_train, Y_train)

Y_pred = model.predict(X_test)

#Comparing Predicted vs Actual Values
print("Predicted values ", np.round(Y_pred[:5], 2))
print("Real values      ", Y_test[:5])
print("Trained W        ", round(model.a[0], 2))
print("Trained b        ", round(model.b, 2))

#Plotting Graph to see visually the dataset + predictions
plt.scatter(X_test, Y_test, color="g")
plt.plot(X_test, Y_pred, color="b")
plt.title("Rented bike count vs Hour")
plt.xlabel("Hour")
plt.ylabel("Number of Rented Bikes")
plt.show()
```

```
Predicted values  [1213.72 1165.62  973.23 1069.42  588.44]
Real values       [ 242 1115 2235  158  696]
Trained W         48.1
Trained b         107.45
```



As shown from the graph, the data doesn't have a clear trend due to being such a large dataset with lots of noise. From preventing overfitting, one could argue our graph is now underfitted, suggesting we should use a more complex model to analyse this huge, complicated data set.

Notes about this submission

You will submit a single Jupyter notebook for this project assignment. Details will be provided for this on the MA124 Moodle page.

- The last thing you should do before submitting the notebook is to Restart Kernel and Run All Cells. You should then save the notebook and submit the .ipynb file. **You will lose marks if you submit a notebook that has not been run.**
- You are expected to add code and markdown cells to this document as appropriate to provide your responses to the tasks. Instructions about this are given throughout but feel free to add markdown cells at any point to provide clarity or comments.
- Likewise, to help the reader, please provide appropriate comments in your code (for example functions or blocks of code should have comments about what they do, variables should be described in comments, as appropriate).